

ACM_COMPETITION_TEMPLATE

算法竞赛板子

按模块打印，按需复制

板子数量：36

生成时间：2026-09-10 10:51

说明：PDF 直接从当前 .cpp 板子生成，Markdown 仅作为目录索引。

板子目录

基础

- BasicAlgo.cpp
- BinarySearch.cpp
- Sortings.cpp
- BigInteger.cpp

数据结构

- DataStructure/CartesianTree.cpp
- DataStructure/ChairmanTree.cpp
- DataStructure/FenwickTree.cpp
- DataStructure/HLD.cpp
- DataStructure/SegmentTree.cpp
- DataStructure/Stack&Queue.cpp
- DataStructure/SSTable.cpp
- DataStructure/Trie.cpp
- DataStructure/UnionFind.cpp

动态规划

- DynamicProgramming/DigitDynamicProgramming.cpp
- DynamicProgramming/KackPack.cpp

图论

- Graph/BipartiteGraph.cpp
- Graph/Dinic.cpp
- Graph/Hierholzer.cpp
- Graph/KruskalTree.cpp
- Graph/LCA.cpp
- Graph/MinSpanningTree.cpp
- Graph/ShortestPath.cpp
- Graph/Tarjan.cpp
- Graph/TopoSort.cpp

数学

- Math/Comb.cpp
- Math/LinearBase.cpp
- Math/Math.cpp
- Math/MatrixPow.cpp
- Math/Mu.cpp

字符串

- String/ACAutomaton.cpp
- String/ExKMP.cpp
- String/Hash.cpp
- String/KMP.cpp
- String/Manacher.cpp
- String/MinimalRotation.cpp
- String/SA.cpp

BasicAlgo.cpp

```

1  #include <bits/stdc++.h>
2  using namespace std;
3  #define int long long
4  const int N = 1005;
5  int a[N][N], pre[N][N], diff[N][N]; // 均为 1-based
6  // 二维前缀和, O(nm)
7  void buildPrefix(int n, int m){
8      for(int i = 1; i <= n; i++)
9          for(int j = 1; j <= m; j++)
10             pre[i][j] = pre[i - 1][j] + pre[i][j - 1]
11                 - pre[i - 1][j - 1] + a[i][j];
12 }
13 int queryPrefix(int x1, int y1, int x2, int y2){
14     return pre[x2][y2] - pre[x1 - 1][y2]
15         - pre[x2][y1 - 1] + pre[x1 - 1][y1 - 1];
16 }
17 // 矩形 [x1,x2] * [y1,y2] 加 val
18 void addRect(int x1, int y1, int x2, int y2, int val){
19     diff[x1][y1] += val;
20     diff[x1][y2 + 1] -= val;
21     diff[x2 + 1][y1] -= val;
22     diff[x2 + 1][y2 + 1] += val;
23 }
24 // 还原差分数组, 结果仍在 diff 中, O(nm)
25 void buildDiff(int n, int m){
26     for(int i = 1; i <= n; i++)
27         for(int j = 1; j <= m; j++)
28             diff[i][j] += diff[i - 1][j] + diff[i][j - 1]
29                 - diff[i - 1][j - 1];
30 }

```

BinarySearch.cpp

```

1  namespace Searching {
2      // 整数二分查找 (寻找左边界)
3      // check(mid): 检查 mid 是否满足条件
4      // 区间 [l, r] 被划分成 [l, mid] 和 [mid + 1, r] 时使用
5      int binarySearchLeft(int l, int r, const function<bool(int)> &check) {
6          while (l < r) {
7              int mid = (l + r) >> 1;
8              if (check(mid)) r = mid;
9              else
10                 l = mid + 1;
11          }
12          return l;
13      }
14      // 整数二分查找 (寻找右边界)
15      // 区间 [l, r] 被划分成 [l, mid - 1] 和 [mid, r] 时使用
16      int binarySearchRight(int l, int r, const function<bool(int)> &check) {
17          while (l < r) {
18              int mid = (l + r + 1) >> 1;
19              if (check(mid)) l = mid;
20              else
21                 r = mid - 1;
22          }
23          return l;
24      }
25      // 实数二分查找
26      // eps: 精度要求, 通常比题目要求多 2 位
27      double binarySearchReal(double l, double r, const function<bool(double)> &check, double eps =
1e-8) {
28          while (r - l > eps) {

```

```

29             double mid = (l + r) / 2;
30             if (check(mid)) r = mid;
31             else
32                 l = mid;
33         }
34         return l;
35     }
36 }

```

Sortings.cpp

```

1  #include <bits/stdc++.h>
2  using namespace std;
3  #define int long long
4  const int N = 200005;
5  int a[N], tmp[N]; // 1-based
6  // 归并排序, O(n log n)
7  // 归并排序, 按升序排列全局数组 a[l..r]。
8  void mergeSort(int l, int r){
9      if(l >= r) return;
10     int mid = (l + r) >> 1;
11     mergeSort(l, mid);
12     mergeSort(mid + 1, r);
13     int i = l, j = mid + 1, k = l;
14     while(i <= mid && j <= r){
15         if(a[i] <= a[j]) tmp[k++] = a[i++];
16         else tmp[k++] = a[j++];
17     }
18     while(i <= mid) tmp[k++] = a[i++];
19     while(j <= r) tmp[k++] = a[j++];
20     for(int p = l; p <= r; p++) a[p] = tmp[p];
21 }
22 // 快速排序, 平均 O(n log n)
23 // 快速排序, 按升序排列全局数组 a[l..r]。
24 void quickSort(int l, int r){
25     if(l >= r) return;
26     int i = l - 1, j = r + 1, x = a[(l + r) >> 1];
27     while(i < j){
28         do i++; while(a[i] < x);
29         do j--; while(a[j] > x);
30         if(i < j) swap(a[i], a[j]);
31     }
32     quickSort(l, j);
33     quickSort(j + 1, r);
34 }

```

BigInteger.cpp

```

1  // 高精度非负整数模板: 用低位在前的数组保存每一位数字。
2  namespace BigInteger {
3      // 将十进制字符串转换成低位在前的数字数组, 例如 "123" -> {3,2,1}。
4      vector<int> toVec(const string &s) {
5          vector<int> A;
6          for (int i = (int)s.size() - 1; i >= 0; i--)
7              A.push_back(s[i] - '0');
8          while (A.size() > 1 && A.back() == 0)
9              A.pop_back();
10         return A;
11     }
12     // 按正常的十进制顺序输出低位在前的数字数组。
13     void printVec(const vector<int> &A) {
14         for (int i = (int)A.size() - 1; i >= 0; i--)
15             cout << A[i];
16     }

```

```

17 // 比较两个非负高精度整数, 判断 A 是否大于等于 B.
18 bool cmp(const vector<int> &A, const vector<int> &B) {
19     if (A.size() != B.size()) return A.size() > B.size();
20     for (int i = (int)A.size() - 1; i >= 0; i--)
21         if (A[i] != B[i]) return A[i] > B[i];
22     return true;
23 }
24 // 计算两个非负高精度整数 A + B.
25 vector<int> add(const vector<int> &A, const vector<int> &B) {
26     if (B.size() > A.size()) return add(B, A);
27     vector<int> C;
28     int t = 0;
29     for (int i = 0; i < (int)A.size(); i++) {
30         t += A[i];
31         if (i < (int)B.size()) t += B[i];
32         C.push_back(t % 10);
33         t = t > 9 ? 1 : 0;
34     }
35     if (t) C.push_back(1);
36     return C;
37 }
38 // 计算 A - B, 要求 A >= B 且 A、B 均为非负数.
39 vector<int> sub(const vector<int> &A, const vector<int> &B) {
40     vector<int> C;
41     int t = 0;
42     for (int i = 0; i < (int)A.size(); i++) {
43         t = A[i] - t;
44         if (i < (int)B.size()) t -= B[i];
45         C.push_back((t + 10) % 10);
46         t = t < 0 ? 1 : 0;
47     }
48     while (C.size() > 1 && C.back() == 0)
49         C.pop_back();
50     return C;
51 }
52 // 计算非负高精度整数 A 与普通非负整数 b 的乘积.
53 vector<int> mul(const vector<int> &A, int b) {
54     vector<int> C;
55     long long t = 0;
56     for (int i = 0; i < (int)A.size() || t; i++) {
57         if (i < (int)A.size()) t += 1LL * A[i] * b;
58         C.push_back((int)(t % 10));
59         t /= 10;
60     }
61     while (C.size() > 1 && C.back() == 0)
62         C.pop_back();
63     return C;
64 }
65 // 计算 A / b, 要求 b > 0, 并将余数写入 r.
66 vector<int> div(const vector<int> &A, int b, int &r) {
67     vector<int> C;
68     r = 0;
69     for (int i = (int)A.size() - 1; i >= 0; i--) {
70         r = r * 10 + A[i];
71         C.push_back(r / b);
72         r %= b;
73     }
74     reverse(C.begin(), C.end());
75     while (C.size() > 1 && C.back() == 0)
76         C.pop_back();
77     return C;
78 }
79 }

```

DataStructure/CartesianTree.cpp

```

1 #include <bits/stdc++.h>
2 using namespace std;
3 #define int long long
4 const int N = 200005; int a[N], ls[N], rs[N], stk[N]; int top, root;
5 // 小根笛卡尔树, ls/rs 为左右儿子, O(n)
6 // 按数组下标建立小根笛卡尔树, 结果保存在 ls、rs 和 root 中.
7 void buildCartesian(int n){
8     fill(ls + 1, ls + n + 1, 0);
9     fill(rs + 1, rs + n + 1, 0);
10    top = root = 0;
11    for(int i = 1; i <= n; i++){
12        int last = 0;
13        while(top && a[stk[top]] > a[i]) last = stk[top--];
14        ls[i] = last;
15        if(top) rs[stk[top]] = i;
16        else root = i;
17        stk[++top] = i;
18    }
19 }

```

DataStructure/ChairmanTree.cpp

```

1 #include <bits/stdc++.h>
2 using namespace std;
3 const int N = 1e6 + 5; const int MAXNODE = N * 25;
4 struct Node {
5     int val, lc, rc;
6 } tree[MAXNODE];
7 int tot, n, m; int a[N], roots[N];
8 // 建立初始版本的主席树, 返回覆盖区间 [l, r] 的根节点.
9 int build(int l, int r){
10     int root = ++tot;
11     if(l == r){
12         tree[root].val = a[l];
13         return root;
14     }
15     int mid = (l + r) / 2;
16     tree[root].lc = build(l, mid);
17     tree[root].rc = build(mid + 1, r);
18     return root;
19 }
20 // 从旧版本 pre 复制并修改单点 pos, 返回新版本根节点.
21 int update(int pre, int l, int r, int pos, int val){
22     int root = ++tot;
23     tree[root] = tree[pre];
24     if(l == r){
25         tree[root].val = val;
26         return root;
27     }
28     int mid = (l + r) / 2;
29     if(pos <= mid){
30         tree[root].lc = update(tree[pre].lc, l, mid, pos, val);
31     }
32     else{
33         tree[root].rc = update(tree[pre].rc, mid + 1, r, pos, val);
34     }
35     return root;
36 }
37 // 查询某个主席树版本在位置 pos 上保存的值.
38 int query(int root, int l, int r, int pos){
39     if(l == r) return tree[root].val;
40     int mid = (l + r) / 2;

```

```

41     if (pos <= mid) return query(tree[root].lc,1,mid,pos);
42     else
43         return query(tree[root].rc,mid + 1,r,pos);
44 }
45 // 可持久化线段树: 支持基于历史版本的单点修改和单点查询。
46 void solve(){
47     int n,m;
48     cin >> n >> m;
49     for(int i = 1;i <= n;i++){
50         cin >> a[i];
51         roots[0] = build(1,n);
52         for(int i = 1;i <= m;i++){
53             int v,t,p,c;
54             cin >> v >> t;
55             if(t == 1){
56                 cin >> p >> c;
57                 roots[i] = update(roots[v],1,n,p,c);
58             }
59             else{
60                 cin >> p;
61                 cout << query(roots[v],1,n,p) << endl;
62                 roots[i] = roots[v];
63             }
64         }
65     }
66     // k-th
67     const int N = 2e5 + 5;
68     struct Node{
69         int l,r; int sum;
70     }tree[N << 5];
71     int root[N]; int tot;
72     // 将离散值 pos 插入到主席树新版本 now 中。
73     void insert(int pre,int &now,int l,int r,int pos){
74         now = ++tot;
75         tree[now] = tree[pre];
76         tree[now].sum++;
77         if (l == r) return;
78         int mid = (l + r) >> 1;
79         if (pos <= mid) insert(tree[pre].l,tree[now].l,1,mid,pos);
80         else
81             insert(tree[pre].r,tree[now].r,mid + 1,r,pos);
82     }
83     // 在版本差 root[v]-root[u] 中查询第 k 小的离散值。
84     int query(int u,int v,int l,int r,int k){
85         if (l == r) return l;
86         int cnt = tree[tree[v].l].sum - tree[tree[u].l].sum; int mid = (l + r) >> 1;
87         if (k <= cnt) return query(tree[u].l,tree[v].l,1,mid,k);
88         else
89             return query(tree[u].r,tree[v].r,mid + 1,r,k - cnt);
90     }
91     // 查询两个版本在单个离散位置 k 上的计数差。
92     int query2(int u,int v,int l,int r,int k){
93         if (l == r) return tree[u].sum - tree[v].sum;
94         int mid = (l + r) >> 1;
95         if (k <= mid) return query2(tree[u].l,tree[v].l,1,mid,k);
96         else
97             return query2(tree[u].r,tree[v].r,mid + 1,r,k);
98     }
99     // 主席树求区间第 k 小: 用 root[r]-root[l-1] 表示区间频数。
100    void solve(){
101        int n,m;
102        cin >> n >> m;
103        vector<int> a(n + 1);

```

```

104        vector<int> dis;
105        for(int i = 1;i <= n;i++){
106            cin >> a[i];
107            dis.push_back(a[i]);
108        }
109        sort(all(dis));
110        dis.erase(unique(all(dis)),dis.end());
111        auto get = [&](int x){
112            return lower_bound(all(dis),x) - dis.begin() + 1;
113        };
114        int len = sz(dis);
115        for(int i = 1;i <= n;i++){
116            insert(root[i - 1],root[i],1,len,get(a[i]));
117        }
118        while(m--){
119            int l,r,k;
120            cin >> l >> r >> k;
121            int pos = query(root[l - 1],root[r],1,len,k);
122            cout << dis[pos - 1] << endl;
123        }

```

DataStructure/FenwickTree.cpp

```

1  #include<bits/stdc++.h>
2  using namespace std;
3  namespace FenwickTree {
4      const int N = 200005; int tree[N]; int n;
5      // 返回 x 的最低位 1, 用于跳转到树状数组的父节点。
6      int lowbit(int x) {
7          return x & (-x);
8      }
9      // 单点增加 d, 维护一维前缀和。
10     void add(int index, int d) {
11         for (int i = index; i <= n; i += lowbit(i)) {
12             tree[i] += d;
13         }
14     }
15     // 查询一维前缀 [1,index] 的和。
16     int ask(int index) {
17         int sum = 0;
18         for (int i = index; i > 0; i -= lowbit(i)) {
19             sum += tree[i];
20         }
21         return sum;
22     }
23     // 查询一维闭区间 [left,right] 的和。
24     int query(int left, int right) {
25         return ask(right) - ask(left - 1);
26     }
27 }
28 namespace FenwickTree2D {
29     // 二维树状数组 (2D Binary Indexed Tree)
30     // 支持单点修改和矩形区域查询
31     const int N = 505; int tree[N][N]; int n, m;
32     // 初始化
33     // 初始化二维树状数组的行列范围并清空数据。
34     void init(int rows, int cols) {
35         n = rows;
36         m = cols;
37         for (int i = 0; i <= n; i++)
38             for (int j = 0; j <= m; j++)
39                 tree[i][j] = 0;
40     }
41     // 单点更新: 在 (x, y) 位置加上 d

```

```

42 // 将二维点 (x,y) 增加 d.
43 void add(int x, int y, int d) {
44     for (int i = x; i <= n; i += lowbit(i)) {
45         for (int j = y; j <= m; j += lowbit(j)) {
46             tree[i][j] += d;
47         }
48     }
49 }
50 // 查询前缀和: [1,1] 到 (x,y)
51 // 查询二维前缀 [1,x] * [1,y] 的和.
52 int ask(int x, int y) {
53     int sum = 0;
54     for (int i = x; i > 0; i -= lowbit(i)) {
55         for (int j = y; j > 0; j -= lowbit(j)) {
56             sum += tree[i][j];
57         }
58     }
59     return sum;
60 }
61 // 查询矩形区域和: [x1,y1] 到 [x2,y2]
62 // 查询二维闭矩形 [x1,x2] * [y1,y2] 的和.
63 int query(int x1, int y1, int x2, int y2) {
64     return ask(x2, y2) - ask(x1 - 1, y2) - ask(x2, y1 - 1) + ask(x1 - 1, y1 - 1);
65 }
66 }
67 // 使用树状数组离线查询第k小的元素
68 // 需要先对元素进行离散化, 更新时在离散化后的索引位置更新, 查询时根据树状数组的前缀和来找到第k小的元素位置
69 namespace FenwickTreeKth {
70     const int N = 1e5 + 5; int tree[N];
71     // 在线性树状数组中将离散值 index 增加 d.
72     void update(int index, int d, int n) {
73         for (int i = index; i <= n; i += lowbit(i)) {
74             tree[i] += d;
75         }
76     }
77     // 查询第 k 小的元素位置
78     // 用树状数组二进制跳跃查找第 k 小值的离散下标.
79     int find(int k, int n) {
80         int pos = 0;
81         for (int i = 1 << 20; i > 0; i >= 1) {
82             int next_pos = pos + i;
83             if (next_pos <= n && tree[next_pos] < k) {
84                 pos = next_pos;
85                 k -= tree[pos];
86             }
87         }
88         return pos + 1;
89     }
90 }
91 // 示例: 读入频次数组并回答若干个第 k 小查询.
92 void solve(){
93     // 示例用法
94     int n; // 离散化后的元素个数
95     cin >> n;
96     vector<int> arr(n);
97     for (int i = 0; i < n; i++) {
98         cin >> arr[i];
99     }
100     update(arr[i], 1, n); // 更新树状数组
101 }
102 int q; // 查询次数
103 cin >> q;
104 while (q--) {
105     int k;
106     cin >> k;

```

```

105     int pos = find(k, n); // 查询第 k 小的元素位置
106     cout << pos << endl; // 输出结果
107 }
108 }
109 }

```

DataStructure/HLD.cpp

```

1 #include <bits/stdc++.h>
2 using namespace std;
3 #define int long long
4 #define endl "\n"
5 #define all(x) (x).begin(), (x).end()
6 #define rall(x) (x).rbegin(), (x).rend()
7 #define sz(x) (int)(x).size()
8 const int INF = (1LL << 62); const int MOD = 1e9 + 7; const int mod = 998244353; int M;
9 struct Node{
10     int sum = 0; int mn = INF; int mx = -INF;
11 };
12 // 合并两段线段树信息, 维护区间和、最小值和最大值.
13 Node merge(Node a, Node b){
14     Node res;
15     res.sum = (a.sum + b.sum) % M;
16     res.mn = min(a.mn, b.mn);
17     res.mx = max(a.mx, b.mx);
18     return res;
19 }
20 struct SegmentTreeAdd{
21     int n = 0;
22     vector<Node> tree;
23     vector<int> lazy;
24     vector<int> a;
25     // 用 dfs 序数组初始化线段树.
26     void init(const vector<int> &arr){
27         n = (int)arr.size() - 1;
28         a.assign(n + 1, 0);
29         for(int i = 1; i <= n; i++) a[i] = arr[i];
30         tree.assign(4 * n + 5, Node());
31         lazy.assign(4 * n + 5, 0);
32         if(n) build(1, n, 1);
33     }
34     // 建立线段树节点 p, 覆盖 dfs 序区间 [l,r].
35     void build(int l, int r, int p){
36         if(l == r){
37             tree[p] = {a[l] % M, a[l], a[l]};
38             return;
39         }
40         int mid = (l + r) >> 1;
41         build(l, mid, p << 1);
42         build(mid + 1, r, p << 1 | 1);
43         pushUp(p);
44     }
45     // 用两个儿子重新计算节点 p 的信息.
46     void pushUp(int p){
47         tree[p] = merge(tree[p << 1], tree[p << 1 | 1]);
48     }
49     // 给节点 p 覆盖区间整体加 v, 并记录懒标记.
50     void apply(int p, int v, int len){
51         tree[p].sum += v * len;
52         tree[p].sum %= M;
53         tree[p].mn += v;
54         tree[p].mx += v;
55         lazy[p] += v;
56         lazy[p] %= M;

```

```

57     }
58     // 将节点 p 的区间加标记下传给两个儿子。
59     void pushDown(int p,int ln,int rn){
60         if(lazy[p] == 0)return;
61         apply(p << 1,lazy[p],ln);
62         apply(p << 1 | 1,lazy[p],rn);
63         lazy[p] = 0;
64     }
65     // 对 dfs 序区间 [ql,qr] 整体加 v。
66     void add(int ql,int qr,int v){
67         if(ql > qr || !n)return;
68         add(ql,qr,v,1,n,1);
69     }
70     // 线段树递归实现的区间加辅助函数。
71     void add(int ql,int qr,int v,int l,int r,int p){
72         if(ql <= l && r <= qr){
73             apply(p,v,r - l + 1);
74             return;
75         }
76         int mid = (l + r) >> 1;
77         pushDown(p,mid - l + 1,r - mid);
78         if(ql <= mid)add(ql,qr,v,l,mid,p << 1);
79         if(qr > mid)add(ql,qr,v,mid + 1,r,p << 1 | 1);
80         pushUp(p);
81     }
82     // 查询 dfs 序区间 [ql,qr] 的聚合信息。
83     Node query(int ql,int qr){
84         if(ql > qr || !n)return Node();
85         return query(ql,qr,1,n,1);
86     }
87     // 线段树递归实现的区间查询辅助函数。
88     Node query(int ql,int qr,int l,int r,int p){
89         if(ql <= l && r <= qr)return tree[p];
90         int mid = (l + r) >> 1;
91         pushDown(p,mid - l + 1,r - mid);
92         if(qr <= mid)return query(ql,qr,l,mid,p << 1);
93         if(ql > mid)return query(ql,qr,mid + 1,r,p << 1 | 1);
94         return merge(
95             query(ql,qr,l,mid,p << 1),
96             query(ql,qr,mid + 1,r,p << 1 | 1)
97         );
98     }
99 };
100 // 树链剖分模板: 支持路径加、路径和、子树加和子树和。
101 void solve(){
102     int n,m,r,p;cin >> n >> m >> r >> M;
103     vector<int> val(n + 1);
104     vector<vector<int>> adj(n + 1);
105     for(int i = 1;i <= n;i++)cin >> val[i];
106     for(int i = 0;i < n - 1;i++){
107         int u,v;cin >> u >> v;
108         adj[u].push_back(v);
109         adj[v].push_back(u);
110     }
111     vector<int> fa(n + 1),dfn(n + 1),son(n + 1),top(n + 1),
112         siz(n + 1,1),dep(n + 1),rev(n + 1);
113     function<void(int,int)> dfs1,dfs2;
114     dfs1 = [&](int u,int p){
115         int mx = 0;
116         for(auto v : adj[u]){
117             if(v == p)continue;
118             fa[v] = u;
119             dep[v] = dep[u] + 1;

```

```

120             dfs1(v,u);
121             siz[u] += siz[v];
122             if(siz[v] > mx){
123                 son[u] = v;
124                 mx = max(mx,siz[v]);
125             }
126         }
127     };
128     int timer = 0;
129     dfs2 = [&](int u,int tp){
130         top[u] = tp;
131         dfn[u] = ++timer;
132         rev[timer] = u;
133         if (son[u] != 0) dfs2(son[u],tp);
134         for(auto v : adj[u]){
135             if(v == fa[u] || v == son[u])continue;
136             dfs2(v,v);
137         }
138     };
139     dfs1(r,0);dfs2(r,r);
140     vector<int> base(n + 1);
141     for(int i = 1;i <= n;i++){
142         base[dfn[i]] = val[i];
143     }
144     SegmentTreeAdd seg;
145     seg.init(base);
146     function<void(int,int,int)> tadd;
147     tadd = [&](int x,int y,int z){
148         while(top[x] != top[y]){
149             if (dep[top[x]] < dep[top[y]]) swap(x,y);
150             seg.add(dfn[top[x]],dfn[x],z);
151             x = fa[top[x]];
152         }
153         if (dep[x] > dep[y]) swap(x,y);
154         seg.add(dfn[x],dfn[y],z);
155     };
156     function<int(int,int)> tquery;
157     tquery = [&](int x,int y){
158         int ans = 0;
159         while(top[x] != top[y]){
160             if (dep[top[x]] < dep[top[y]]) swap(x,y);
161             ans += seg.query(dfn[top[x]],dfn[x]).sum;
162             x = fa[top[x]];
163         }
164         if (dep[x] > dep[y]) swap(x,y);
165         ans += seg.query(dfn[x],dfn[y]).sum;
166         return ans;
167     };
168     for(int i = 0;i < m;i++){
169         int t;cin >> t;
170         if(t == 1){
171             int x,y,z;cin >> x >> y >> z;
172             tadd(x,y,z);
173         }
174         if(t == 2){
175             int x,y;cin >> x >> y;
176             cout << tquery(x,y) % M << endl;
177         }
178         if(t == 3){
179             int x,z;cin >> x >> z; int l = dfn[x],r = dfn[x] + siz[x] - 1;
180             seg.add(l,r,z);
181         }
182         if(t == 4){

```

```

183         int x;cin >> x; int l = dfn[x],r = dfn[x] + siz[x] - 1;
184         cout << seg.query(l,r).sum % M << endl;
185     }
186 }
187 }

```

DataStructure/SegmentTree.cpp

```

1  #include <bits/stdc++.h>
2  using namespace std;
3  const int INF = (1LL << 62);
4  /*
5   这份文件里的线段树按模块复制，不需要整份全抄。
6   当前第一个模块 SegmentTreeAdd 支持：区间加、区间赋值、区间和、区间最小值、区间最大值。
7   使用它时需要一起复制 Node、merge 和 SegmentTreeAdd。
8   如果题目没有区间赋值，只保留区间加：可以删掉 applySet、lazySet、hasSet 和两个 modify 函数。
9   */
10 struct Node{
11     int sum = 0;int mn = INF;int mx = -INF;
12 };
13 // 合并两个区间的信息，维护区间和、最小值和最大值。
14 Node merge(const Node &a,const Node &b){
15     Node res;
16     res.sum = a.sum + b.sum;
17     res.mn = min(a.mn,b.mn);
18     res.mx = max(a.mx,b.mx);
19     return res;
20 }
21 // ===== 模块一：区间加 / 区间赋值 / 区间查询 =====
22 // Node 保存一段区间的和、最小值、最大值；lazySet 和 lazyAdd 是两类懒标记。
23 struct SegmentTreeAdd{
24     int n = 0;
25     vector<int> a;
26     vector<Node> tree;
27     vector<int> lazyAdd,lazySet,hasSet;
28     // ----- 初始化与建树：所有用法都需要 -----
29     // 用 1-based 数组初始化支持区间加、区间赋值和区间查询的线段树。
30     void init(const vector<int> &arr){
31         n = (int)arr.size() - 1;
32         a = arr;
33         tree.assign(4 * n + 5, Node());
34         lazyAdd.assign(4 * n + 5, 0);
35         lazySet.assign(4 * n + 5, 0);
36         hasSet.assign(4 * n + 5, 0);
37         if(n)build(1,n,1);
38     }
39     // 建立节点 p，覆盖区间 [l,r]。
40     void build(int l,int r,int p){
41         if(l == r){
42             tree[p] = {a[l],a[l],a[l]};
43             return;
44         }
45         int mid = (l + r) >> 1;
46         build(l,mid,p << 1);
47         build(mid + 1,r,p << 1 | 1);
48         pushUp(p);
49     }
50     // ----- 维护与懒标记：所有区间修改都需要 -----
51     // 根据两个儿子更新节点 p 的区间信息。
52     void pushUp(int p){
53         tree[p] = merge(tree[p << 1],tree[p << 1 | 1]);
54     }
55     // 将节点 p 覆盖区间整体赋值为 v，并覆盖已有懒标记。
56     void applySet(int p,int v,int len){

```

```

57         tree[p].sum = v * len;
58         tree[p].mn = v;
59         tree[p].mx = v;
60         lazySet[p] = v;
61         hasSet[p] = 1;
62         lazyAdd[p] = 0;
63     }
64     // 将节点 p 覆盖区间整体加 v。
65     void applyAdd(int p,int v,int len){
66         tree[p].sum += v * len;
67         tree[p].mn += v;
68         tree[p].mx += v;
69         lazyAdd[p] += v;
70     }
71     // 将赋值标记和加法标记下传给左右儿子。
72     void pushDown(int p,int ln,int rn){
73         if(hasSet[p]){
74             applySet(p << 1,lazySet[p],ln);
75             applySet(p << 1 | 1,lazySet[p],rn);
76             hasSet[p] = 0;
77         }
78         if(lazyAdd[p]){
79             applyAdd(p << 1,lazyAdd[p],ln);
80             applyAdd(p << 1 | 1,lazyAdd[p],rn);
81             lazyAdd[p] = 0;
82         }
83     }
84     // ----- 区间加：需要区间加时复制这一组 -----
85     // 对闭区间 [ql,qr] 整体加 v。
86     void add(int ql,int qr,int v){
87         if(ql > qr || !n)return;
88         add(ql,qr,v,1,n,1);
89     }
90     // 区间加的递归实现，节点 p 覆盖 [l,r]。
91     void add(int ql,int qr,int v,int l,int r,int p){
92         if(ql <= l && r <= qr){
93             applyAdd(p,v,r - l + 1);
94             return;
95         }
96         int mid = (l + r) >> 1;
97         pushDown(p,mid - l + 1,r - mid);
98         if(ql <= mid)add(ql,qr,v,l,mid,p << 1);
99         if(qr > mid)add(ql,qr,v,mid + 1,r,p << 1 | 1);
100        pushUp(p);
101    }
102    // ----- 区间赋值：不需要赋值时可以连同 lazySet 一起删掉 -----
103    // 对闭区间 [ql,qr] 整体赋值为 v。
104    void modify(int ql,int qr,int v){
105        if(ql > qr || !n)return;
106        modify(ql,qr,v,1,n,1);
107    }
108    // 区间赋值的递归实现，节点 p 覆盖 [l,r]。
109    void modify(int ql,int qr,int v,int l,int r,int p){
110        if(ql <= l && r <= qr){
111            applySet(p,v,r - l + 1);
112            return;
113        }
114        int mid = (l + r) >> 1;
115        pushDown(p,mid - l + 1,r - mid);
116        if(ql <= mid)modify(ql,qr,v,l,mid,p << 1);
117        if(qr > mid)modify(ql,qr,v,mid + 1,r,p << 1 | 1);
118        pushUp(p);
119    }

```

```

120 // ----- 区间查询: 需要查询时复制这一组 -----
121 // 查询闭区间 [ql,qr] 的和、最小值和最大值。
122 Node query(int ql,int qr){
123     if(ql > qr || !n)return Node();
124     return query(ql,qr,l,n,1);
125 }
126 // 区间查询的递归实现, 节点 p 覆盖 [l,r]。
127 Node query(int ql,int qr,int l,int r,int p){
128     if(ql <= l && r <= qr)return tree[p];
129     int mid = (l + r) >> 1;
130     pushDown(p,mid - 1 + 1,r - mid);
131     if(qr <= mid)return query(ql,qr,l,mid,p << 1);
132     if(ql > mid)return query(ql,qr,mid + 1,r,p << 1 | 1);
133     return merge(
134         query(ql,qr,l,mid,p << 1),
135         query(ql,qr,mid + 1,r,p << 1 | 1)
136     );
137 }
138 };
139 // ===== 模块二: 模意义区间加 / 区间乘 / 区间和 =====
140 // 与上面的 SegmentTreeAdd 无关, 只需要模运算时复制这个结构体即可。
141 struct SegmentTreeMul{
142     int n = 0; int mod = 1;
143     vector<int> a,sum,lazyAdd,lazyMul;
144     // 初始化支持区间加、区间乘和区间求和的模运算线段树。
145     void init(const vector<int> &arr,int modValue){
146         n = (int)arr.size() - 1;
147         mod = modValue;
148         a.assign(n + 1,0);
149         for(int i = 1;i <= n;i++)a[i] = arr[i];
150         sum.assign(4 * n + 5,0);
151         lazyAdd.assign(4 * n + 5,0);
152         lazyMul.assign(4 * n + 5,1);
153         if(n)build(1,n,1);
154     }
155     // 将 x 规范到 [0,mod) 内。
156     int norm(int x){
157         x %= mod;
158         if(x < 0)x += mod;
159         return x;
160     }
161     // 建立模线段树节点 p, 覆盖区间 [l,r]。
162     void build(int l,int r,int p){
163         if(l == r){
164             sum[p] = norm(a[l]);
165             return;
166         }
167         int mid = (l + r) >> 1;
168         build(l,mid,p << 1);
169         build(mid + 1,r,p << 1 | 1);
170         pushUp(p);
171     }
172     // 用两个儿子的模和更新父节点。
173     void pushUp(int p){
174         sum[p] = (sum[p << 1] + sum[p << 1 | 1]) % mod;
175     }
176     // 将变换 x -> x*mulv+addv 应用到节点 p。
177     void apply(int p,int addv,int mulv,int len){
178         addv = norm(addv);
179         mulv = norm(mulv);
180         sum[p] = (sum[p] * mulv + addv * len) % mod;
181         lazyMul[p] = lazyMul[p] * mulv % mod;
182         lazyAdd[p] = (lazyAdd[p] * mulv + addv) % mod;
183     }
184     // 下传区间乘加复合懒标记。
185     void pushDown(int p,int ln,int rn){
186         if(lazyAdd[p] == 0 && lazyMul[p] == 1)return;
187         apply(p << 1,lazyAdd[p],lazyMul[p],ln);
188         apply(p << 1 | 1,lazyAdd[p],lazyMul[p],rn);
189         lazyAdd[p] = 0;
190         lazyMul[p] = 1;
191     }
192     // 对闭区间 [ql,qr] 整体加 v (模 mod) 。
193     void add(int ql,int qr,int v){
194         if(ql > qr || !n)return;
195         add(ql,qr,v,1,n,1);
196     }
197     // 模线段树区间加的递归实现。
198     void add(int ql,int qr,int v,int l,int r,int p){
199         if(ql <= l && r <= qr){
200             apply(p,v,1,r - l + 1);
201             return;
202         }
203         int mid = (l + r) >> 1;
204         pushDown(p,mid - 1 + 1,r - mid);
205         if(ql <= mid)add(ql,qr,v,l,mid,p << 1);
206         if(qr > mid)add(ql,qr,v,mid + 1,r,p << 1 | 1);
207         pushUp(p);
208     }
209     // 对闭区间 [ql,qr] 整体乘 v (模 mod) 。
210     void mul(int ql,int qr,int v){
211         if(ql > qr || !n)return;
212         mul(ql,qr,v,1,n,1);
213     }
214     // 模线段树区间乘的递归实现。
215     void mul(int ql,int qr,int v,int l,int r,int p){
216         if(ql <= l && r <= qr){
217             apply(p,0,v,r - l + 1);
218             return;
219         }
220         int mid = (l + r) >> 1;
221         pushDown(p,mid - 1 + 1,r - mid);
222         if(ql <= mid)mul(ql,qr,v,l,mid,p << 1);
223         if(qr > mid)mul(ql,qr,v,mid + 1,r,p << 1 | 1);
224         pushUp(p);
225     }
226     // 查询闭区间 [ql,qr] 的元素和 (模 mod) 。
227     int query(int ql,int qr){
228         if(ql > qr || !n)return 0;
229         return query(ql,qr,1,n,1);
230     }
231     // 模线段树区间查询的递归实现。
232     int query(int ql,int qr,int l,int r,int p){
233         if(ql <= l && r <= qr)return sum[p];
234         int mid = (l + r) >> 1;
235         pushDown(p,mid - 1 + 1,r - mid);
236         int ans = 0;
237         if(ql <= mid)ans = (ans + query(ql,qr,l,mid,p << 1)) % mod;
238         if(qr > mid)ans = (ans + query(ql,qr,mid + 1,r,p << 1 | 1)) % mod;
239         return ans;
240     }
241 };
242 namespace DynamicSegmentTree
243 {
244     const int MAXN = 3000000; int ls[MAXN], rs[MAXN], sum[MAXN], lazyAdd[MAXN]; int cnt = 0;
245     // 创建一个动态线段树节点并返回其编号。

```

```

246 int newNode()
247 {
248     return ++cnt;
249 }
250 // 根据左右子节点更新动态节点 i 的区间和。
251 void pushUp(int i)
252 {
253     sum[i] = (ls[i] ? sum[ls[i]] : 0) + (rs[i] ? sum[rs[i]] : 0);
254 }
255 // 将增量 v 应用到动态节点 i 覆盖的区间 [l,r]。
256 void apply(int i, int v, int l, int r)
257 {
258     if (!i) return;
259     sum[i] += v * (r - l + 1);
260     lazyAdd[i] += v;
261 }
262 // 创建需要的子节点，并下传动态线段树懒标记。
263 void pushDown(int i, int l, int r)
264 {
265     if (lazyAdd[i] == 0) return;
266     int mid = (l + r) >> 1;
267     if (!ls[i]) ls[i] = newNode();
268     if (!rs[i]) rs[i] = newNode();
269     apply(ls[i], lazyAdd[i], l, mid);
270     apply(rs[i], lazyAdd[i], mid + 1, r);
271     lazyAdd[i] = 0;
272 }
273 // 动态开点线段树的区间加操作。
274 void add(int ql, int qr, int v, int l, int r, int &i)
275 {
276     if (!i) i = newNode();
277     if (ql <= l && r <= qr){
278         sum[i] += v * (r - l + 1);
279         lazyAdd[i] += v;
280         return;
281     }
282     pushDown(i, l, r);
283     int mid = (l + r) >> 1;
284     if (ql <= mid) add(ql, qr, v, l, mid, ls[i]);
285     if (qr > mid) add(ql, qr, v, mid + 1, r, rs[i]);
286     pushUp(i);
287 }
288 // 动态开点线段树的区间和查询。
289 int query(int ql, int qr, int l, int r, int i)
290 {
291     if (!i) return 0;
292     if (ql <= l && r <= qr) return sum[i];
293     pushDown(i, l, r);
294     int mid = (l + r) >> 1; int ans = 0;
295     if (ql <= mid) ans += query(ql, qr, l, mid, ls[i]);
296     if (qr > mid) ans += query(ql, qr, mid + 1, r, rs[i]);
297     return ans;
298 }
299 }
300 // zkw SegmentTree max
301 const int N = 600005;
302 int tree[N << 2]; int siz;
303 // 建立 ZKW 线段树，维护数组的区间最大值。
304 void build(vector<int> &a){
305     int n = (int)a.size() - 1;
306     siz = 1;
307     while(siz < n) siz <= 1;
308     for(int i = 0; i < (siz << 1) + 5; i++)

```

```

309     tree[i] = -INF;
310     for(int i = 1; i <= n; i++){
311         tree[siz + i - 1] = a[i];
312         for(int i = siz - 1; i--){
313             tree[i] = max(tree[i << 1], tree[i << 1 | 1]);
314         }
315     } // 将 ZKW 线段树中的单点 pos 增加 val。
316     void add(int pos, int val){
317         pos += siz - 1;
318         tree[pos] += val;
319         while(pos >= 1)
320             tree[pos] = max(tree[pos << 1], tree[pos << 1 | 1]);
321     }
322     // 查询 ZKW 线段树区间 [l,r] 的最大值。
323     int query(int l, int r)
324     {
325         int ans = -INF;
326         l += siz - 1; r += siz - 1;
327         while(l <= r)
328         {
329             if (l & 1) ans = max(ans, tree[l++]);
330             if (!(r & 1)) ans = max(ans, tree[r--]);
331             l >>= 1; r >>= 1;
332         }
333         return ans;
334     }
335     // 在线段树上寻找下标不小于 l 且值至少为 x 的第一个位置。
336     int find(int l, int x, int node, int L, int R){
337         if (R < l || tree[node] < x) return -1;
338         if (L == R) return L;
339         int mid = (L + R) >> 1;
340         int res = find(l, x, node << 1, L, mid);
341         if (res != -1) return res;
342         return find(l, x, node << 1 | 1, mid + 1, R);
343     }
344     // 查询整个 ZKW 线段树中满足条件的第一个位置。
345     int find(int l, int x){
346         return find(l, x, 1, 1, siz);
347     }

```

DataStructure/Stack&Queue.cpp

```

1 #include <bits/stdc++.h>
2 using namespace std;
3 #define int long long
4 const int N = 200005; int a[N],nxt[N],windowMax[N]; int stk[N],que[N],top,head,tail;
5 // nxt[i]: 右侧第一个严格更大元素, O(n)
6 // 求每个位置右侧第一个严格更大的元素下标。
7 void nextGreater(int n){
8     top = 0;
9     for(int i = n; i >= 1; i--){
10         while(top && a[stk[top]] <= a[i])top--;
11         nxt[i] = top ? stk[top] : -1;
12         stk[++top] = i;
13     }
14 }
15 // windowMax[1..n-k+1], O(n)
16 // 用单调队列求所有长度为 k 的滑动窗口最大值。
17 void slidingWindowMax(int n, int k){
18     head = tail = 1;
19     for(int i = 1; i <= n; i++){
20         while(head < tail && que[head] <= i - k)head++;
21         while(head < tail && a[que[tail - 1]] <= a[i])tail--;
22         que[tail++] = i;

```

```

23     if(i >= k)windowMax[i - k + 1] = a[que[head]];
24 }
25 }

```

DataSet/STTable.cpp

```

1 #include <bits/stdc++.h>
2 using namespace std;
3 #define int long long
4 const int N = 200005; const int LOG = 20; int a[N],lg2[N]; int stMax[N][LOG],stMin[N][LOG];
5 // 静态区间最值预处理, O(n log n)
6 // 预处理静态数组的区间最大值和最小值。
7 void initST(int n){
8     lg2[1] = 0;
9     for(int i = 2;i <= n;i++)lg2[i] = lg2[i >> 1] + 1;
10    for(int i = 1;i <= n;i++)stMax[i][0] = stMin[i][0] = a[i];
11    for(int j = 1;j < LOG;j++){
12        for(int i = 1;i + (1LL << j) - 1 <= n;i++){
13            stMax[i][j] = max(stMax[i][j - 1],stMax[i + (1LL << (j - 1))][j - 1]);
14            stMin[i][j] = min(stMin[i][j - 1],stMin[i + (1LL << (j - 1))][j - 1]);
15        }
16    }
17    // 查询闭区间 [l,r] 的最大值。
18    int queryMax(int l, int r){
19        int k = lg2[r - l + 1];
20        return max(stMax[l][k],stMax[r - (1LL << k) + 1][k]);
21    }
22    // 查询闭区间 [l,r] 的最小值。
23    int queryMin(int l, int r){
24        int k = lg2[r - l + 1];
25        return min(stMin[l][k],stMin[r - (1LL << k) + 1][k]);
26    }

```

DataSet/Trie.cpp

```

1 #include <cstring>
2 #include <string>
3 using namespace std;
4 namespace Trie {
5     const int N = 100000; int tr[N][26], endCnt[N], idx;
6     void init() {
7         memset(tr, 0, sizeof(tr));
8         memset(endCnt, 0, sizeof(endCnt));
9         idx = 1; // 1 号点作为根
10    }
11    void insert(const string &s) {
12        int p = 1;
13        for (char c : s) {
14            int u = c - 'a';
15            if (!tr[p][u]) tr[p][u] = ++idx;
16            p = tr[p][u];
17        }
18        endCnt[p]++;
19    }
20    int countWord(const string &s) {
21        int p = 1;
22        for (char c : s) {
23            int u = c - 'a';
24            if (!tr[p][u]) return 0;
25            p = tr[p][u];
26        }
27        return endCnt[p];
28    }
29    int countPrefix(const string &s) {

```

```

30    int p = 1; int sum = 0;
31    for (char c : s) {
32        int u = c - 'a';
33        if (!tr[p][u]) break;
34        p = tr[p][u];
35        sum += endCnt[p];
36    }
37    return sum;
38    }
39    namespace ZeroOneTrie {
40        const int B = 31;int tr[N][2], idx;
41        void init() {
42            memset(tr, 0, sizeof(tr));
43            idx = 1;
44        }
45        void insert(int x) {
46            int p = 1;
47            for (int i = B; i >= 0; i--) {
48                int u = (x >> i) & 1;
49                if (!tr[p][u]) tr[p][u] = ++idx;
50                p = tr[p][u];
51            }
52        }
53        // 返回与 x 异或后的最大值
54        int maxXor(int x) {
55            int p = 1;int res = 0;
56            for (int i = B; i >= 0; i--) {
57                int u = (x >> i) & 1; int want = u ^ 1;
58                if (tr[p][want]) {
59                    res = (res << 1) | 1;
60                    p = tr[p][want];
61                } else {
62                    res <<= 1;
63                    p = tr[p][u];
64                }
65            }
66            return res;
67        }
68    }
69 }

```

DataSet/UnionFind.cpp

```

1 #include <bits/stdc++.h>
2 using namespace std;
3 #define int long long
4 const int N = 200005; int fa[N];
5 int siz[N],rankValue[N];
6 // 初始化 1..n 的并查集, 每个点初始为独立集合。
7 void init(int n){
8     iota(fa + 1,fa + n + 1,1);
9     fill(siz + 1,siz + n + 1,1);
10    fill(rankValue + 1,rankValue + n + 1,0);
11 }
12 // 查找 x 所在集合的代表, 并进行路径压缩。
13 int find(int x){
14     return fa[x] == x ? x : fa[x] = find(fa[x]);
15 }
16 // 合并 x、y 所在集合; 若原本已连通则返回 false。
17 bool merge(int x, int y){
18     x = find(x); y = find(y);
19     if(x == y)return false;
20     if(rankValue[x] < rankValue[y])swap(x,y);
21     fa[y] = x;

```

```

22     siz[x] += siz[y];
23     if(rankValue[x] == rankValue[y])rankValue[x]++;
24     return true;
25 }
26 // 返回 x 所在连通块的大小。
27 int getSize(int x){
28     return siz[find(x)];
29 }

```

DynamicProgramming/DigitDynamicProgramming.cpp

```

1  #include <bits/stdc++.h>
2  using namespace std;
3  // Windy' s number
4  // 统计区间 [a,b] 内相邻数字差至少为 2 的数字个数。
5  void solve(){
6      int a,b;
7      cin >> a >> b;
8      vector<int> num(20);
9      vector<vector<int>> dp(20,vector<int>(15,-1));
10     auto dfs = [&](auto dfs,int pos,int last,bool limit,bool zero) -> int {
11         if(pos == 0)return 1;
12         if (!zero && !limit && dp[pos][last] != -1) return dp[pos][last];
13         int res = 0 ; int lim = (limit)?num[pos] : 9;
14         for(int i = 0;i <= lim;i++){
15             if(!zero && abs(i - last) < 2)continue;
16             res += dfs(dfs,pos - 1,(zero && i == 0)?0:i,limit && i == lim,zero && i == 0);
17         }
18         if (!limit && !zero) dp[pos][last] = res;
19         return res;
20     };
21     auto cal = [&](int x){
22         int len = 0;
23         while(x){
24             num[++len]=x%10;
25             x/=10;
26         }
27         return dfs(dfs,len,0,1,1);
28     };
29     cout << cal(b) - cal(a - 1) << endl;
30 }
31 signed main(){
32     ios::sync_with_stdio(0); cin.tie(0);
33     #ifdef LOCAL
34         freopen("in.txt", "r", stdin);
35         freopen("out.txt", "w", stdout);
36     #endif
37     int t = 1;
38     // cin >> t;
39     while(t--> solve());
40     return 0;
41 }

```

DynamicProgramming/KackPack.cpp

```

1  #include <bits/stdc++.h>
2  using namespace std;
3  #define int long long
4  const int N = 200005; const int M = 200005;
5  int w[N],v[N],c[N],dp[M]; // 1-based
6  vector<pair<int,int>> group[N];
7  // 将容量为 0..W 的背包状态清零。
8  void clearDP(int W){
9     fill(dp,dp + W + 1,0);

```

```

10 }
11 // dp[j] 表示容量不超过 j 的最大价值
12 // 处理每件物品最多选一次的 0/1 背包。
13 void zeroOneKnapsack(int n, int W){
14     clearDP(W);
15     for(int i = 1;i <= n;i++){
16         for(int j = W;j >= w[i];j--){
17             dp[j] = max(dp[j],dp[j - w[i]] + v[i]);
18         }
19     }
20     // 处理每件物品可以选任意次的完全背包。
21     void completeKnapsack(int n, int W){
22         clearDP(W);
23         for(int i = 1;i <= n;i++){
24             for(int j = w[i];j <= W;j++){
25                 dp[j] = max(dp[j],dp[j - w[i]] + v[i]);
26             }
27         }
28     }
29     // 多重背包，二进制拆分
30     // 用二进制拆分处理每件物品有数量上限的多重背包。
31     void multipleKnapsack(int n, int W){
32         clearDP(W);
33         for(int i = 1;i <= n;i++){
34             int rest = c[i];
35             for(int k = 1;k <= rest;k <= 1){
36                 int nw = k * w[i],nv = k * v[i];
37                 for(int j = W;j >= nw;j--){
38                     dp[j] = max(dp[j],dp[j - nw] + nv);
39                 }
40                 rest -= k;
41             }
42         }
43     }
44     // 第 i 组放在 group[i] 中，每组最多选一个
45     // 处理每组最多选一个物品的分组背包。
46     void groupedKnapsack(int n, int W){
47         clearDP(W);
48         for(int i = 1;i <= n;i++){
49             for(int j = W;j >= 0;j--){
50                 for(auto [weight,value] : group[i]){
51                     if (j >= weight) dp[j] = max(dp[j],dp[j - weight] + value);
52                 }
53             }
54         }
55     }

```

Graph/BipartiteGraph.cpp

```

1  #include <bits/stdc++.h>
2  using namespace std;
3  namespace BipartiteColoring {
4      const int SIZE = 100000 + 10;
5      vector<int> edge[SIZE];
6      int color[SIZE]; // -1: uncolored, 0/1: two parts
7      // 清空前 n 个点的邻接表，并将颜色恢复为未染色。
8      void init(int n) {
9          for (int i = 1; i <= n; i++) edge[i].clear();
10         memset(color, -1, sizeof(color));
11     }
12     // 添加一条无向边，用于二分图染色判定。
13     void addEdge(int u, int v) {
14         edge[u].push_back(v);
15         edge[v].push_back(u);
16     }
17     // 从 u 开始染色；若发现同色相邻点则返回 false。

```

```

18 bool dfs(int u, int c) {
19     color[u] = c;
20     for (auto v : edge[u]) {
21         if (color[v] == -1) {
22             if (!dfs(v, c ^ 1)) return false;
23         }
24         else if (color[v] == c) {
25             return false;
26         }
27     }
28     return true;
29 }
30 // 检查 1..n 的整个图是否为二分图。
31 bool check(int n) {
32     for (int i = 1; i <= n; i++) {
33         if (color[i] == -1) {
34             if (!dfs(i, 0)) return false;
35         }
36     }
37     return true;
38 }
39 }
40 // matching
41 // 用增广路算法求二分图最大匹配数。
42 void solve() {
43     int n, m, e; cin >> n >> m >> e;
44     vector<vector<int>> adj(n + m + 1);
45     for (int i = 0; i < e; i++) {
46         int u, v;
47         cin >> u >> v;
48         adj[u].push_back(v);
49     }
50     vector<int> vis(n + m + 1);
51     vector<int> match(n + m + 1);
52     auto find = [&](auto find, int u) -> bool {
53         for (auto v : adj[u]) {
54             if (vis[v]) continue;
55             vis[v] = 1;
56             if (match[v] == 0 || find(find, match[v])) {
57                 match[v] = u;
58                 return 1;
59             }
60         }
61         return 0;
62     };
63     int cnt = 0;
64     for (int i = 1; i <= n; i++) {
65         vis.assign(n + m + 1, 0);
66         if (find(find, i)) cnt++;
67     }
68     cout << cnt << endl;
69 }

```

Graph/Dinic.cpp

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 const int INF = (1LL << 62);
5 struct Edge {
6     int v, cap, rev;
7 };
8 struct Dinic {
9     int n, s, t;

```

```

10     vector<vector<Edge>> adj;
11     vector<int> dep, cur;
12     // 初始化 n 个点、源点 s 和汇点 t 的网络流图。
13     Dinic(int n, int s, int t) : n(n), s(s), t(t) {
14         adj.resize(n + 1); dep.resize(n + 1); cur.resize(n + 1);
15     }
16     // 添加容量为 w 的有向边，同时建立反向边。
17     void addEdge(int u, int v, int w) {
18         Edge a = {v, w, (int)adj[v].size()};
19         Edge b = {u, 0, (int)adj[u].size()};
20         adj[u].push_back(a);
21         adj[v].push_back(b);
22     }
23     // 在残量网络上分层，判断汇点是否仍然可达。
24     bool bfs() {
25         fill(all(dep), -1);
26         queue<int> q; q.push(s);
27         dep[s] = 0;
28         while (!q.empty()) {
29             int u = q.front();
30             q.pop();
31             for (auto e : adj[u]) {
32                 if (e.cap > 0 && dep[e.v] == -1) {
33                     dep[e.v] = dep[u] + 1;
34                     q.push(e.v);
35                 }
36             }
37         }
38         return dep[t] != -1;
39     }
40     // 沿分层图从 u 推送不超过 flow 的阻塞流。
41     int dfs(int u, int flow) {
42         if (u == t) return flow;
43         for (int &i = cur[u]; i < sz(adj[u]); i++) {
44             Edge &e = adj[u][i];
45             if (e.cap <= 0) continue;
46             if (dep[e.v] != dep[u] + 1) continue;
47             int x = dfs(e.v, min(flow, e.cap));
48             if (x == 0) continue;
49             e.cap -= x;
50             adj[e.v][e.rev].cap += x;
51             return x;
52         }
53         return 0;
54     }
55     // 不断构造分层图并增广，返回最大流。
56     int maxFlow() {
57         int ans = 0;
58         while (bfs()) {
59             fill(all(cur), 0);
60             while (int flow = dfs(s, INF)) {
61                 ans += flow;
62             }
63         }
64         return ans;
65     }
66 };
67 // 读入有向容量网络并输出最大流。
68 void solve() {
69     int n, m, s, t; cin >> n >> m >> s >> t;
70     Dinic dinic(n, s, t);
71     for (int i = 0; i < m; i++) {
72         int u, v, w; cin >> u >> v >> w;

```

```

73     dinic.addEdge(u,v,w);
74 }
75 cout << dinic.maxFlow() << endl;
76 }

```

Graph/Hierholzer.cpp

```

1  #include <bits/stdc++.h>
2  using namespace std;
3
4  #define all(x) (x).begin(), (x).end()
5  #define rall(x) (x).rbegin(), (x).rend()
6  #define sz(x) (int)(x).size()
7  // 构造有向图的欧拉路径; 不存在时输出 No.
8  void solve(){
9      int n,m;cin >> n >> m;
10     vector<vector<int>> adj(n + 1);
11     vector<int> in(n + 1),out(n + 1);
12     for(int i = 0;i < m;i++){
13         int u,v;
14         cin >> u >> v;
15         adj[u].push_back(v);
16         in[v]++;out[u]++;
17     }
18     for(int i = 1;i <= n;i++){
19         sort(all(adj[i]));
20     }
21     int start = -1; int cnt_start = 0,cnt_end = 0;
22     for(int i = 1;i <= n;i++){
23         if(in[i] + 1 == out[i]){
24             start = i;
25             cnt_start++;
26         }
27         else if(in[i] == out[i] + 1){
28             cnt_end++;
29         }
30         else if(in[i] != out[i]){
31             cout << "No" << endl;
32             return;
33         }
34     }
35     if(!((cnt_start == 0 && cnt_end == 0) || (cnt_start == 1 && cnt_end == 1))){
36         cout << "No" << endl;
37         return;
38     }
39     if(start == -1){
40         for(int i = 1;i <= n;i++){
41             if(out[i]){
42                 start = i;
43                 break;
44             }
45         }
46     }
47     if(start == -1){
48         cout << 1 << endl;
49         return;
50     }
51     vector<int> it(n + 1),path;
52     auto dfs = [&](auto dfs,int u) -> void {
53         while(it[u] < sz(adj[u])){
54             int v = adj[u][it[u]];
55             it[u]++;
56             dfs(dfs,v);
57         }

```

```

58         path.push_back(u);
59     };
60     dfs(dfs,start);
61     if(sz(path) != m + 1){
62         cout << "No" << endl;
63         return;
64     }
65     reverse(all(path));
66     for(auto i : path){
67         cout << i << " ";
68     }
69     cout << endl;
70 }

```

Graph/KruskalTree.cpp

```

1  #include <bits/stdc++.h>
2  using namespace std;
3  #define int long long
4  const int N = 200005; const int LOG = 20;
5  struct Edge{
6      int u,v,w;
7  };
8  int fa[N]; int val[N];
9  // 查找并查集代表, 用于合并 Kruskal 树节点.
10 int find(int x){
11     if (fa[x]!=x) fa[x]=find(fa[x]);
12     return fa[x];
13 }
14 vector<int> tree[N];
15 // 按边权从大到小建最大 Kruskal 重构树.
16 void solve(){
17     int n,m;
18     cin>>n>>m;
19     vector<Edge> edges(m);
20     for(auto &[u,v,w]:edges){
21         cin>>u>>v>>w;
22     }
23     sort(edges.begin(),edges.end(),[](Edge a,Edge b){
24         return a.w>b.w; // 最大生成树
25     });
26     for(int i=1;i<=2*n;i++){
27         fa[i]=i;
28     }
29     int tot=n;
30     for(auto [u,v,w]:edges){
31         int fu=find(u); int fv=find(v);
32         if(fu==fv)continue;
33         ++tot;
34         val[tot]=w;
35         tree[tot].push_back(fu);
36         tree[tot].push_back(fv);
37         fa[fu]=tot;fa[fv]=tot;fa[tot]=tot;
38     }
39     // 找根
40     int root=find(1);
41 }

```

Graph/LCA.cpp

```

1  #include <bits/stdc++.h>
2  using namespace std;
3  const int N = 200005; const int LOG = 20;
4  vector<int> adj[N];
5  int fa[N][LOG]; int dep[N];

```

```

6 // 预处理普通 LCA 的祖先表和节点深度。
7 void dfs(int u,int f){
8     fa[u][0] = f;
9     dep[u] = dep[f] + 1;
10    for(int i = 1;i < LOG;i++){
11        fa[u][i] = fa[fa[u][i-1]][i-1];
12    }
13    for(auto v:adj[u]){
14        if (v == f) continue;
15        dfs(v,u);
16    }
17 }
18 // 用倍增求两个节点的最近公共祖先。
19 int lca(int x,int y){
20     if (dep[x] < dep[y]) swap(x,y);
21     // 把 x 跳到和 y 同一深度
22     for(int i = LOG-1;i >= 0;i--){
23         if (dep[fa[x][i]] >= dep[y]) x = fa[x][i];
24     }
25     if (x == y) return x;
26     // 一起向上跳
27     for(int i = LOG-1;i >= 0;i--){
28         if(fa[x][i] != fa[y][i]){
29             x = fa[x][i];
30             y = fa[y][i];
31         }
32     }
33     return fa[x][0];
34 }
35 // 读入一棵树并回答若干次普通 LCA 查询。
36 void solve(){
37     int n;
38     cin >> n;
39     for(int i = 1;i < n;i++){
40         int u,v;
41         cin >> u >> v;
42         adj[u].push_back(v);
43         adj[v].push_back(u);
44     }
45     dfs(1,0);
46     int q;
47     cin >> q;
48     while(q--){
49         int x,y;
50         cin >> x >> y;
51         cout << lca(x,y) << endl;
52     }
53 }
54 struct EDGE {
55     int v,w;
56 };
57 vector<EDGE> adj[N];
58 int fa[N][LOG]; int mx[N][LOG]; int dep[N];
59 // 预处理带边权树的祖先表及向上路径最大边权。
60 void dfs(int u,int f,int w){
61     fa[u][0] = f;
62     mx[u][0] = w;
63     dep[u] = dep[f] + 1;
64     for(int i = 1;i < LOG;i++){
65         fa[u][i] = fa[fa[u][i-1]][i-1];
66         mx[u][i] = max(mx[u][i-1],mx[fa[u][i-1]][i-1]);
67     }
68     for(auto [v,w]:adj[u]){

```

```

69         if (v == f) continue;
70         dfs(v,u,w);
71     }
72 }
73 // 查询两点路径上的最大边权。
74 int qmx(int x,int y){
75     int ans = 0;
76     if (dep[x] < dep[y]) swap(x,y);
77     // 把 x 跳到和 y 同一深度
78     for(int i = LOG-1;i >= 0;i--){
79         if(dep[fa[x][i]] >= dep[y]){
80             ans = max(ans,mx[x][i]);
81             x = fa[x][i];
82         }
83     }
84     if (x == y) return ans;
85     // 一起向上跳
86     for(int i = LOG-1;i >= 0;i--){
87         if(fa[x][i] != fa[y][i]){
88             ans = max(ans,mx[x][i]);
89             ans = max(ans,mx[y][i]);
90             x = fa[x][i];
91             y = fa[y][i];
92         }
93     }
94     ans = max(ans,mx[x][0]);
95     ans = max(ans,mx[y][0]);
96     return ans;
97 }

```

Graph/MinSpanningTree.cpp

```

1 #include <bits/stdc++.h>
2 using namespace std;
3 #define int long long
4 const int N = 200005; const int M = 400005;
5 struct Edge{ int u,v,w; } edge[M];
6 int fa[N];
7 vector<pair<int,int>> graph[N];
8 // 查找并查集代表, 并进行路径压缩。
9 int find(int x){
10     return fa[x] == x ? x : fa[x] = find(fa[x]);
11 }
12 // 合并两个连通块, 返回是否真的发生了合并。
13 bool merge(int x, int y){
14     x = find(x); y = find(y);
15     if(x == y) return false;
16     fa[x] = y;
17     return true;
18 }
19 // edge[0..m-1] 用 1..n 编号, O(m log m)
20 // 用 Kruskal 算法求最小生成树, 不连通时返回 -1。
21 int kruskal(int n, int m){
22     for(int i = 1;i <= n;i++) fa[i] = i;
23     sort(edge,edge + m,[&](Edge a,Edge b){ return a.w < b.w; });
24     int ans = 0,cnt = 0;
25     for(int i = 0;i < m;i++){
26         if (merge(edge[i].u,edge[i].v)) ans += edge[i].w,cnt++;
27     }
28     return cnt == n - 1 ? ans : -1;
29 }
30 // 向 Prim 使用的无向邻接表添加一条带权边。
31 void addEdge(int u, int v, int w){
32     graph[u].push_back({v,w});
33     graph[v].push_back({u,w});
34 }

```

```

33 }
34 // 从 start 出发用 Prim 算法求最小生成树, 不连通时返回 -1.
35 int prim(int n, int start = 1){
36     vector<int> vis(n + 1);
37     priority_queue<pair<int,int>,vector<pair<int,int>>,greater<pair<int,int>>> q;
38     q.push({0,start});
39     int ans = 0,cnt = 0;
40     while(!q.empty()){
41         auto [w,u] = q.top(); q.pop();
42         if(vis[u])continue;
43         vis[u] = 1; ans += w; cnt++;
44         for(auto [v,nw] : graph[u])
45             if(!vis[v])q.push({nw,v});
46     }
47     return cnt == n ? ans : -1;
48 }

```

Graph/ShortestPath.cpp

```

1 #include <bits/stdc++.h>
2 using namespace std;
3 namespace ShortestPath {
4 #define int long long
5
6 const int N = 200005;
7 const int NN = 505;
8 const int INF = (1LL << 62);
9
10 /*
11  按需复制:
12  1. Dijkstra / SPFA: 复制 Edge, adj, initadj, addEdge 和对应算法。
13  2. Bellman-Ford: 复制 BellmanEdge, bellmanEdge 及其初始化和算法。
14  3. Floyd: 只复制 floydDis, initFloyd, addFloydEdge, floyd。
15 */
16
17 // ===== Dijkstra / SPFA: vector 邻接表 =====
18 struct Edge{ int v,w; };
19 vector<Edge> adj[N];
20 int dis[N];
21 // 清空 1..n 的 vector 邻接表。
22 void initadj(int n){
23     for(int i = 1;i <= n;i++)adj[i].clear();
24 }
25 // 添加一条有向带权边 u -> v.
26 void addEdge(int u,int v,int w){
27     adj[u].push_back({v,w});
28 }
29 // Dijkstra: 处理非负边权, 结果写入 dis[1..n].
30 void dijkstra(int n,int s){
31     fill(dis + 1,dis + n + 1,INF);
32     priority_queue<pair<int,int>,vector<pair<int,int>>,greater<pair<int,int>>> q;
33     dis[s] = 0;
34     q.push({0,s});
35     while(!q.empty()){
36         auto [d,u] = q.top(); q.pop();
37         if(d != dis[u])continue;
38         for(auto [v,w] : adj[u]){
39             if(dis[v] > dis[u] + w){
40                 dis[v] = dis[u] + w;
41                 q.push({dis[v],v});
42             }
43         }
44     }
45 }

```

```

46
47 // SPFA: 支持负边, 并检测源点可达的负环。
48 bool spfa(int n,int s){
49     fill(dis + 1,dis + n + 1,INF);
50     vector<int> inQueue(n + 1),cnt(n + 1);
51     queue<int> q;
52     dis[s] = 0;
53     q.push(s);
54     inQueue[s] = 1;
55     while(!q.empty()){
56         int u = q.front(); q.pop();
57         inQueue[u] = 0;
58         for(auto [v,w] : adj[u]){
59             if(dis[v] > dis[u] + w){
60                 dis[v] = dis[u] + w;
61                 if(!inQueue[v]){
62                     q.push(v);
63                     inQueue[v] = 1;
64                     if(++cnt[v] > n)return false;
65                 }
66             }
67         }
68     }
69     return true;
70 }
71
72 // ===== Bellman-Ford: vector 边集 =====
73 struct BellmanEdge{ int u,v,w; };
74 vector<BellmanEdge> bellmanEdge;
75 // 清空 Bellman-Ford 的边集。
76 void initBellman(){
77     bellmanEdge.clear();
78 }
79 // 向 Bellman-Ford 的边集添加一条有向边。
80 void addBellmanEdge(int u,int v,int w){
81     bellmanEdge.push_back({u,v,w});
82 }
83 // 求单源最短路, 并返回是否不存在源点可达的负环。
84 bool bellmanFord(int n,int s){
85     fill(dis + 1,dis + n + 1,INF);
86     dis[s] = 0;
87     for(int i = 1;i <= n;i++){
88         bool changed = false;
89         for(auto [u,v,w] : bellmanEdge){
90             if(dis[u] != INF && dis[v] > dis[u] + w){
91                 dis[v] = dis[u] + w;
92                 changed = true;
93             }
94         }
95         if(!changed)return true;
96     }
97     return false;
98 }
99
100 // ===== Floyd: 全点对最短路 =====
101 int floydDis[NN][NN];
102 // 初始化 Floyd 距离矩阵, 点自身距离为 0, 其余为 INF.
103 void initFloyd(int n){
104     for(int i = 1;i <= n;i++)
105         for(int j = 1;j <= n;j++)
106             floydDis[i][j] = i == j ? 0 : INF;
107 }
108 // 加入 Floyd 的有向边, 平行边只保留最小权值。

```

```

109 void addFloydEdge(int u,int v,int w){
110     floydDis[u][v] = min(floydDis[u][v],w);
111 }
112 // Floyd-Warshall: 计算所有点对之间的最短路。
113 void floyd(int n){
114     for(int k = 1;k <= n;k++){
115         for(int i = 1;i <= n;i++){
116             if(floydDis[i][k] == INF)continue;
117             for(int j = 1;j <= n;j++){
118                 if(floydDis[k][j] == INF)continue;
119                 floydDis[i][j] = min(floydDis[i][j],floydDis[i][k] + floydDis[k][j]);
120             }
121         }
122     }
123 }

```

Graph/Tarjan.cpp

```

1  #include <bits/stdc++.h>
2  using namespace std;
3  // Tarjan 求割点: 输出无向图中删除后会增加连通块的顶点。
4  struct Edge{
5      int v,id;
6  };
7  void solve(){
8      int n,m;cin >> n >> m;
9      vector<vector<Edge>> adj(n + 1);
10     for(int i = 1;i <= m;i++){
11         int u, v;cin >> u >> v;
12         adj[u].push_back({v,i});
13         adj[v].push_back({u,i});
14     }
15     vector<int> dfn(n + 1),low(n + 1),cut(n + 1);
16     int p = 0;
17     function<void(int,int)> tarjan;
18     tarjan = [&](int u,int pe){
19         dfn[u] = low[u] = ++p;
20         int child = 0;
21         for(auto [v,id] : adj[u]){
22             if(id == pe)continue;
23             if(!dfn[v]){
24                 child++;
25                 tarjan(v,id);
26                 low[u] = min(low[u],low[v]);
27                 if (pe != -1 && low[v] >= dfn[u]) cut[u] = 1;
28             }
29             else{
30                 low[u] = min(low[u],dfn[v]);
31             }
32         }
33         if (pe == -1 && child >= 2) cut[u] = 1;
34     };
35     for(int i = 1;i <= n;i++){
36         if (!dfn[i]) tarjan(i,-1);
37     }
38     int cnt = 0;
39     for(int i = 1;i <= n;i++){
40         cnt += cut[i];
41     }
42     cout << cnt << endl;
43     for(int i = 1;i <= n;i++){
44         if (cut[i]) cout << i << " ";
45     }cout << endl;
46 }

```

```

47 // Tarjan 缩点并在 DAG 上求从 1 到 n 的最大点权路径和。
48 void solve(){
49     int n,m;cin >> n >> m;
50     vector<int> a(n + 1);
51     for(int i = 1;i <= n;i++)cin >> a[i];
52     vector<vector<int>> adj(n + 1);
53     for(int i = 0;i < m;i++){
54         int u,v;cin >> u >> v;
55         adj[u].push_back(v);
56     }
57     vector<int> dfn(n + 1),low(n + 1),in(n + 1),belong(n + 1);
58     vector<int> st;
59     int p = 0;int cnt = 0;
60     function<void(int)> tarjan;
61     tarjan = [&](int u){
62         dfn[u] = low[u] = ++p;
63         st.push_back(u);
64         in[u] = 1;
65         for(auto v : adj[u]){
66             if(!dfn[v]){
67                 tarjan(v);
68                 low[u] = min(low[u],low[v]);
69             }
70             else if(in[v]){
71                 low[u] = min(low[u],dfn[v]);
72             }
73         }
74         if(low[u] == dfn[u]){
75             cnt++;
76             while(1){
77                 int v = st.back();
78                 st.pop_back();
79                 in[v] = 0;
80                 belong[v] = cnt;
81                 if(v == u)break;
82             }
83         }
84     };
85     for(int i = 1;i <= n;i++){
86         if(!dfn[i])tarjan(i);
87     }
88     vector<int> w(cnt + 1);
89     for(int i = 1;i <= n;i++){
90         w[belong[i]] += a[i];
91     }
92     vector<vector<int>> dag(cnt + 1);
93     vector<int> deg(n + 1);
94     for(int u = 1;u <= n;u++){
95         for(auto v : adj[u]){
96             int x = belong[u],y = belong[v];
97             if(x == y)continue;
98             dag[x].push_back(y);
99             deg[y]++;
100         }
101     }
102     vector<int> dp(cnt + 1,-INF);
103     dp[belong[1]] = w[belong[1]];
104     queue<int> q;
105     for(int i = 1;i <= cnt;i++){
106         if (deg[i] == 0) q.push(i);
107     }
108     while(!q.empty()){
109         int u = q.front();

```

```

110     q.pop();
111     for(auto v : dag[u]){
112         if (dp[u] != -INF) dp[v] = max(dp[v],dp[u] + w[v]);
113         deg[v]--;
114         if (deg[v] == 0) q.push(v);
115     }
116 }
117 cout << dp[belong[n]] << endl;
118 }
119 // Tarjan 求点双连通分量, 并输出每个分量包含的顶点.
120 void solve(){
121     int n,m;cin >> n >> m;
122     vector<vector<pair<int,int>>> adj(n + 1);
123     for(int i = 1;i <= m;i++){
124         int u,v;cin >> u >> v;
125         // 自环不增加新的点
126         if(u == v)continue;
127         adj[u].push_back({v,i});
128         adj[v].push_back({u,i});
129     }
130     vector<int> dfn(n + 1),low(n + 1);
131     vector<int> st;
132     vector<vector<int>> ans;
133     int timer = 0;
134     function<void(int,int)> tarjan;
135     tarjan = [&](int u,int inEdge){
136         dfn[u] = low[u] = ++timer;
137         st.push_back(u);
138         for(auto [v,id] : adj[u]){
139             // 不能只判断 v == fa, 因为可能有重边
140             if(id == inEdge)continue;
141             if(!dfn[v]){
142                 tarjan(v,id);
143                 low[u] = min(low[u],low[v]);
144                 if(low[v] >= dfn[u]){
145                     vector<int> cur;
146                     while(1){
147                         int x = st.back();
148                         st.pop_back();
149                         cur.push_back(x);
150                         if(x == v)break;
151                     }
152                     cur.push_back(u);
153                     ans.push_back(cur);
154                 }
155             }
156             else if(dfn[v] < dfn[u]){
157                 // 只处理指向祖先的返祖边
158                 low[u] = min(low[u],dfn[v]);
159             }
160         }
161     };
162     for(int i = 1;i <= n;i++){
163         if(dfn[i])continue;
164         tarjan(i,-1);
165         // 根节点不会被上面的条件弹出
166         if (!st.empty() && st.back() == i) st.pop_back();
167         // 孤立点, 或者只有自环的点
168         if (adj[i].empty()) ans.push_back({i});
169     }
170     cout << ans.size() << endl;
171     for(auto &cur : ans){
172         cout << cur.size() << " ";

```

```

173         for(auto x : cur){
174             cout << x << " ";
175         }
176         cout << endl;
177     }
178 }
179 // Tarjan 求桥并缩成边双连通分量.
180 void solve(){
181     int n,m;cin >> n >> m;
182     vector<vector<pair<int,int>>> adj(n + 1);
183     for(int i = 1;i <= m;i++){
184         int u,v;cin >> u >> v;
185         adj[u].push_back({v,i});
186         adj[v].push_back({u,i});
187     }
188     vector<int> dfn(n + 1),low(n + 1);
189     vector<int> bridge(m + 1);
190     int timer = 0;
191     function<void(int,int)> tarjan;
192     tarjan = [&](int u,int inEdge){
193         dfn[u] = low[u] = ++timer;
194         for(auto [v,id] : adj[u]){
195             if(id == inEdge)continue;
196             if(!dfn[v]){
197                 tarjan(v,id);
198                 low[u] = min(low[u],low[v]);
199                 // 注意: 边双是严格大于
200                 if (low[v] > dfn[u]) bridge[id] = 1;
201             }
202             else if(dfn[v] < dfn[u]){
203                 low[u] = min(low[u],dfn[v]);
204             }
205         }
206     };
207     for(int i = 1;i <= n;i++){
208         if (!dfn[i]) tarjan(i,-1);
209     }
210     vector<int> bel(n + 1);
211     vector<vector<int>> ans;
212     int cnt = 0;
213     function<void(int)> dfs;
214     dfs = [&](int u){
215         bel[u] = cnt;
216         ans.back().push_back(u);
217         for(auto [v,id] : adj[u]){
218             if(bridge[id])continue;
219             if(bel[v])continue;
220             dfs(v);
221         }
222     };
223     for(int i = 1;i <= n;i++){
224         if(bel[i])continue;
225         cnt++;
226         ans.push_back({});
227         dfs(i);
228     }
229     cout << cnt << endl;
230     for(auto &cur : ans){
231         cout << cur.size() << " ";
232         for(auto x : cur){
233             cout << x << " ";
234         }
235         cout << endl;

```

```

236     }
237 }

```

Graph/TopoSort.cpp

```

1  #include <bits/stdc++.h>
2  using namespace std;
3  #define int long long
4  const int N = 200005;
5  vector<int> graph[N], topo;
6  int deg[N];
7  // 清空图、入度数组和上一次得到的拓扑序。
8  void initGraph(int n){
9      topo.clear();
10     for(int i = 1; i <= n; i++) graph[i].clear(), deg[i] = 0;
11 }
12 // 添加有向边 u -> v, 并维护 v 的入度。
13 void addEdge(int u, int v){
14     graph[u].push_back(v);
15     deg[v]++;
16 }
17 // Kahn 算法求拓扑序, 成功时结果保存在 topo 中。
18 bool topoSort(int n){
19     queue<int> q;
20     for(int i = 1; i <= n; i++) if(!deg[i]) q.push(i);
21     while(!q.empty()){
22         int u = q.front(); q.pop();
23         topo.push_back(u);
24         for(int v : graph[u]) if(--deg[v] == 0) q.push(v);
25     }
26     return (int)topo.size() == n;
27 }

```

Math/Comb.cpp

```

1  #include <bits/stdc++.h>
2  using namespace std;
3  // 求 a 在质数模 mod 下的乘法逆元。
4  int inv(int a, int mod = MOD) {
5      return qpow((a % mod + mod) % mod, mod - 2, mod);
6  }
7  // 用递推公式一次预处理 1..n 的逆元。
8  void invs(int n, int mod = MOD) {
9      vector<int> inv(n + 1);
10     inv[1] = 1;
11     for (int i = 2; i <= n; i++) {
12         inv[i] = (1LL * (mod - mod / i) * inv[mod % i] % mod);
13     }
14 }
15 // 组合数预处理: C(n, k) = fac[n] * invfac[k] * invfac[n-k] % mod
16 namespace Comb {
17     vector<int> fac(maxn + 1, 1), invfac(maxn + 1);
18     for (int i = 1; i <= maxn; i++) {
19         fac[i] = (1LL * fac[i - 1] * i % mod);
20     }
21     invfac[maxn] = qpow(fac[maxn], mod - 2, mod);
22     for (int i = maxn; i >= 1; i--) {
23         invfac[i - 1] = (1LL * invfac[i] * i % mod);
24     }
25     // 查询预处理范围内的组合数 C(n, k)。
26     int C(int n, int k) {
27         if (k < 0 || k > n) return 0;
28         if (n > maxn) return 0;
29         return (1LL * fac[n] * invfac[k] % mod * invfac[n - k] % mod);

```

```

30     }
31 }
32 // 卡特兰数
33 // Lucas
34 int n, P;
35 vector<int> fac, invfac;
36 // 在模 P 下计算 a 的 b 次幂。
37 int qpow(int a, int b) {
38     int res = 1;
39     while (b) {
40         if (b & 1) res = res * a % P;
41         a = a * a % P;
42         b >>= 1;
43     }
44     return res;
45 }
46 // 使用阶乘和逆阶乘计算模 P 的组合数。
47 int C(int n, int k) {
48     if (k < 0 || k > n) return 0;
49     return fac[n] * invfac[k] % P * invfac[n - k] % P;
50 }
51 // Lucas 定理求大 n、k 在质数模 P 下的组合数。
52 int Lucas(int n, int k) {
53     if (k > n) return 0;
54     int res = 1;
55     while (n || k) {
56         int ni = n % P; int ki = k % P;
57         if (ki > ni) return 0;
58         res = res * C(ni, ki) % P;
59         n /= P;
60         k /= P;
61     }
62     return res;
63 }
64 // 预处理 Lucas 所需的阶乘和逆阶乘数组。
65 void solve(){
66     cin >> n >> P;
67     int lim = min(n, P - 1);
68     fac.assign(lim + 1, 1);
69     invfac.assign(lim + 1, 1);
70     for (int i = 1; i <= lim; i++)
71         fac[i] = fac[i - 1] * i % P;
72     invfac[lim] = qpow(fac[lim], P - 2);
73     for (int i = lim; i >= 1; i--)
74         invfac[i - 1] = invfac[i] * i % P;
75 }

```

Math/LinearBase.cpp

```

1  #include <bits/stdc++.h>
2  using namespace std;
3  vector<int> p(64), pos(N);
4  // 将带编号 id 的数 x 插入线性基, 并维护编号较新的表示。
5  void insert(int x, int id){
6     for(int i = 62; i >= 0; i--){
7         if((x >> i & 1) == 0) continue;
8         if(!p[i]){
9             p[i] = x; pos[i] = id;
10            return;
11        }
12        if(pos[i] < id){
13            swap(pos[i], id);
14            swap(p[i], x);
15        }

```

```

16     x ^= p[i];
17 }
18 }
19 // 建立线性基并求所有输入数异或出的最大值。
20 void solve(){
21     int n;cin >> n;
22     vector<int> a(n);
23     for(int i = 0;i < n;i++){
24         cin >> a[i];
25         insert(a[i],i);
26     }
27     int ans = 0;
28     for(int i = 62;i >= 0;i--){
29         ans = max(ans,ans^p[i]);
30     }
31     cout << ans << endl;
32 }

```

Math/Math.cpp

```

1 #include <bits/stdc++.h>
2 using namespace std;
3 namespace Math {
4     // 常用质数模数
5     const int MOD = 1000000007;
6     // 求两个整数的最大公约数。
7     int gcd(int a, int b) {
8         return b == 0 ? a : gcd(b, a % b);
9     }
10    // 求两个整数的最小公倍数。
11    int lcm(int a, int b) {
12        return a / gcd(a, b) * b;
13    }
14    // 扩展欧几里得：求 gcd(a,b)，并计算 ax+by=gcd(a,b)。
15    int exgcd(int a, int b, int &x, int &y) {
16        if (b == 0) {
17            x = 1;
18            y = 0;
19            return a;
20        }
21        int g = exgcd(b, a % b, y, x);
22        y -= a / b * x;
23        return g;
24    }
25    // 计算 base^exp mod mod。
26    int power(int base, int exp, int mod) {
27        int res = 1;
28        base %= mod;
29        while (exp > 0) {
30            if (exp % 2 == 1) res = (res * base) % mod;
31            base = (base * base) % mod;
32            exp /= 2;
33        }
34        return res;
35    }
36    // 计算整数 base 的 exp 次幂。
37    int power(int base, int exp) {
38        int res = 1;
39        while (exp > 0) {
40            if (exp % 2 == 1) res *= base;
41            base *= base;
42            exp /= 2;
43        }
44        return res;

```

```

45 }
46 // 埃氏筛
47 namespace Eratosthenes {
48     const int N = 1000001; bool is_prime[N];
49     // 埃氏筛：标记 1..n 是否为质数。
50     void sieve(int n) {
51         memset(is_prime, true, sizeof(is_prime));
52         is_prime[0] = is_prime[1] = false;
53         for (int p = 2; p * p <= n; p++) {
54             if (is_prime[p]) {
55                 for (int i = p * p; i <= n; i += p)
56                     is_prime[i] = false;
57             }
58         }
59     }
60 }
61 // 欧拉筛 (线性筛)
62 namespace Euler {
63     const int N = 1000001; int primes[N]; bool is_prime[N]; int prime_count = 0;
64     // 欧拉线性筛：生成 1..n 内的全部质数。
65     void sieve(int n) {
66         memset(is_prime, true, sizeof(is_prime));
67         is_prime[0] = is_prime[1] = false;
68         for (int i = 2; i <= n; i++) {
69             if (is_prime[i]) primes[prime_count++] = i;
70             for (int j = 0; j < prime_count && i * primes[j] <= n; j++) {
71                 is_prime[i * primes[j]] = false;
72                 if (i % primes[j] == 0) break;
73             }
74         }
75     }
76 }
77 // 欧拉筛 (线性筛) + 欧拉函数
78 namespace EulerPhi {
79     const int N = 1000001; int primes[N]; bool is_prime[N];
80     int phi[N]; // phi[i] 存储 i 的欧拉函数值
81     int prime_count = 0;
82     // 线性筛质数并同时计算每个数的欧拉函数值。
83     void sieve(int n) {
84         memset(is_prime, true, sizeof(is_prime));
85         is_prime[0] = is_prime[1] = false;
86         phi[1] = 1;
87         for (int i = 2; i <= n; i++) {
88             if (is_prime[i]) {
89                 primes[prime_count++] = i;
90                 phi[i] = i - 1;
91             }
92             for (int j = 0; j < prime_count && i * primes[j] <= n; j++) {
93                 is_prime[i * primes[j]] = false;
94                 if (i % primes[j] == 0) {
95                     phi[i * primes[j]] = phi[i] * primes[j];
96                     break;
97                 } else {
98                     phi[i * primes[j]] = phi[i] * (primes[j] - 1);
99                 }
100             }
101         }
102     }
103 }
104 // 单独计算一个数 n 的欧拉函数值 phi(n)。
105 int euler_phi(int n) {
106     int result = n;
107     for (int i = 2; i * i <= n; i++) {

```

```

108         if (n % i == 0) {
109             while (n % i == 0)
110                 n /= i;
111             result -= result / i;
112         }
113     }
114     if (n > 1) result -= result / n;
115     return result;
116 }
117 // 计算大指数模幂时使用欧拉降幂, 指数以十进制字符串给出。
118 // 计算一个数的欧拉函数值, 供扩展欧拉定理使用。
119 int euler_phi(int n) {
120     int res = n;
121     for (int i = 2; i * i <= n; i++) {
122         if (n % i == 0) {
123             while (n % i == 0)
124                 n /= i;
125             res -= res / i;
126         }
127     }
128     if (n > 1) res -= res / n;
129     return res;
130 }
131 // 计算 a^b mod m.
132 int qpow(int a, int b, int m) {
133     int ans = 1 % m;
134     a %= m;
135     while(b){
136         if (b & 1) ans = ans * a % m;
137         a = a * a % m;
138         b >>= 1;
139     }
140     return ans;
141 }
142 // 扩展欧拉定理: 指数很大时求 a^b mod m.
143 void solve(){
144     int a, m; string b; cin >> a >> m >> b; int phi = euler_phi(m);
145     auto get = [&](string s, int mod) {
146         int res = 0;
147         for(char c : s){
148             res = (res * 10 + c - '0') % mod;
149         }
150         return res;
151     };
152     int M = get(b, phi); string sp = to_string(phi); bool big = 0;
153     if(sz(b) != sz(sp)){
154         big = sz(b) > sz(sp);
155     }
156     else{
157         big = b >= sp;
158     }
159     if(big) M += phi;
160     cout << qpow(a, M, m) << endl;
161 }
162 // 中国剩余定理: 合并两两互质的同余方程。
163 using i128 = __int128_t;
164 // 扩展欧几里得, 返回 gcd 并求出一组贝祖系数。
165 int exgcd(int a, int b, int &x, int &y) {
166     if (b == 0) {
167         x = 1;
168         y = 0;
169         return a;
170     }

```

```

171     int g = exgcd(b, a % b, y, x);
172     y -= a / b * x;
173     return g;
174 }
175 // 用中国剩余定理求互质模数方程组的最小非负解。
176 void solve(){
177     int n; cin >> n;
178     vector<int> a(n), b(n);
179     int M = 1;
180     for(int i = 0; i < n; i++){
181         cin >> a[i] >> b[i];
182         M *= a[i];
183     }
184     int ans = 0;
185     for(int i = 0; i < n; i++){
186         int Mi = M / a[i]; int x, y;
187         exgcd(Mi, a[i], x, y);
188         x = (x % a[i] + a[i]) % a[i];
189         ans += (i128)b[i] * Mi % M * x % M;
190         ans %= M;
191     }
192     cout << ans << endl;
193 }
194 // 扩展中国剩余定理: 允许模数不互质, 无法合并时输出 -1.
195 using i128 = __int128_t;
196 // 扩展欧几里得, 供 EXCRT 合并当前方程使用。
197 int exgcd(int a, int b, int &x, int &y) {
198     if (b == 0) {
199         x = 1;
200         y = 0;
201         return a;
202     }
203     int g = exgcd(b, a % b, y, x);
204     y -= a / b * x;
205     return g;
206 }
207 // 逐个合并非互质同余方程, 维护当前解和模数的最小公倍数。
208 void solve(){
209     int n; cin >> n;
210     vector<int> a(n), b(n);
211     int M = 1;
212     for(int i = 0; i < n; i++){
213         cin >> a[i] >> b[i];
214     }
215     int ans = 0;
216     for(int i = 0; i < n; i++){
217         int x, y; int g = exgcd(M, a[i], x, y); int d = b[i] - ans;
218         if(d % g != 0){
219             cout << -1 << endl;
220             return;
221         }
222         int mod2 = a[i] / g;
223         i128 k = (i128)(d / g) * x % mod2;
224         if(k < 0) k += mod2;
225         i128 lcm = (i128)(M / g) * a[i];
226         ans = (int)((i128)ans + (i128)M * k) % lcm;
227         M = (int)lcm;
228     }
229     cout << ans << endl;
230 }

```

Math/MatrixPow.cpp

```

1  #include <bits/stdc++.h>
2  using namespace std;
3  struct Matrix{
4      int n;
5      vector<vector<int>> a;
6      Matrix(int n) : n(n),a(n,vector<int>(n,0)) {}
7      // 在模 MOD 下做加法, 避免矩阵乘法中重复写取模逻辑。
8      inline int add(int x, int y) const { return (x + y) % MOD; }
9      // 在模 MOD 下做乘法。
10     inline int mul(int x, int y) const { return (int)(x * y) % MOD; }
11     // 计算两个同阶矩阵的乘积。
12     Matrix operator*(const Matrix& other) const {
13         Matrix res(n);
14         for(int i = 0; i < n; i++){
15             for(int k = 0; k < n; k++){
16                 if(a[i][k]){
17                     for(int j = 0; j < n; j++){
18                         res.a[i][j] = add(res.a[i][j], mul(a[i][k], other.a[k][j]));
19                     }
20                 }
21             }
22         }
23         return res;
24     }
25     // 二进制快速幂计算当前矩阵的 k 次幂。
26     Matrix pow(int k) const {
27         Matrix res(n), base = *this;
28         for(int i = 0; i < n; i++) res.a[i][i] = 1;
29         while(k){
30             if(k & 1) res = res * base;
31             base = base * base;
32             k >>= 1;
33         }
34         return res;
35     }
36 };
37 // 示例: 用矩阵快速幂计算第 n 个 Fibonacci 数。
38 void solve(){
39     int n; cin >> n; Matrix base(2);
40     base.a[0][0] = 1;
41     base.a[0][1] = 1;
42     base.a[1][0] = 1;
43     base.a[1][1] = 0;
44     Matrix ans = base.pow(n);
45     cout << ans.a[0][1] << endl;
46 }

```

Math/Mu.cpp

```

1  #include <bits/stdc++.h>
2  using namespace std;
3  const int N = 50005; int mu[N], pre[N]; int prime[N], pcnt; bool vis[N];
4  // 线性筛计算莫比乌斯函数 mu, 并预处理其前缀和。
5  void init(){
6      mu[1] = 1;
7      for(int i = 2; i < N; i++){
8          if(!vis[i]){
9              prime[pcnt++] = i;
10             mu[i] = -1;
11         }
12         for(int j = 0; j < pcnt && i * prime[j] < N; j++){
13             int x = i * prime[j];

```

```

14             vis[x] = 1;
15             if(i % prime[j] == 0){
16                 mu[x] = 0;
17                 break;
18             }
19             mu[x] = -mu[i];
20         }
21     }
22     // 求和
23     for(int i = 1; i < N; i++){
24         pre[i] = pre[i - 1] + mu[i];
25     }
26 }
27 // 用整除分块统计  $1 \leq x \leq n, 1 \leq y \leq m$  且  $\gcd(x, y) = 1$  的数对数量。
28 int calc(int n, int m){
29     if(n <= 0 || m <= 0) return 0;
30     int ans = 0;
31     for(int l = 1, r; l <= min(n, m); l = r + 1){
32         int qn = n / l; int qm = m / l;
33         r = min(n / qn, m / qm);
34         ans += (pre[r] - pre[l - 1]) * qn * qm;
35     }
36     return ans;
37 }
38 // 读入若干组 (a, b, d), 统计  $\gcd(x, y) = d$  的对应互质缩放数对。
39 void solve(){
40     init();
41     int n; cin >> n;
42     for(int i = 1; i <= n; i++){
43         int a, b, d; cin >> a >> b >> d;
44         //  $\gcd(x, y) = k \rightarrow \gcd(kx, ky) = 1$ 
45         cout << calc(a / d, b / d) << endl;
46     }
47 }

```

String/ACAutomaton.cpp

```

1  #include <bits/stdc++.h>
2  using namespace std;
3  const int N = 1e6 + 5; int cnt[N]; int tr[N][26]; int fail[N]; int tot;
4  // 插入一个模式串, 并记录它在 Trie 终点出现的次数。
5  void insert(string s){
6      int u = 0;
7      for(auto ch : s){
8          int c = ch - 'a';
9          if (!tr[u][c]) tr[u][c] = ++tot;
10         u = tr[u][c];
11     }
12     cnt[u]++;
13 }
14 // 用 BFS 建立失配指针, 并补齐自动机的转移边。
15 void build(){
16     queue<int> q;
17     for(int c = 0; c < 26; c++){
18         if (tr[0][c]) q.push(tr[0][c]);
19     }
20     while(!q.empty()){
21         int u = q.front();
22         q.pop();
23         for(int c = 0; c < 26; c++){
24             if(tr[u][c]){
25                 fail[tr[u][c]] = tr[fail[u]][c];
26                 q.push(tr[u][c]);
27             }

```

```

28         else{
29             tr[u][c] = tr[fail[u]][c];
30         }
31     }
32 }
33 }
34 // 建立 AC 自动机并扫描文本, 统计所有模式串的匹配次数。
35 void solve(){
36     int n;cin >> n;
37     for(int i = 0;i < n;i++){
38         string s; cin >> s;
39         insert(s);
40     }
41     build();
42     string s;cin >> s; int u = 0;int ans = 0;
43     for(auto ch : s){
44         int c = ch - 'a';
45         u = tr[u][c];
46         for(int p = u;p = fail[p]){
47             if (cnt[p] == -1) break;
48             ans += cnt[p];
49             cnt[p] = -1;
50         }
51     }
52     cout << ans << endl;
53 }

```

String/ExKMP.cpp

```

1  #include<bits/stdc++.h>
2  using namespace std;
3  // 计算字符串自身的 Z 函数, z[i] 表示后缀与整个串的最长公共前缀。
4  vector<int> get_z(string s){
5      int n = sz(s);
6      vector<int> z(n);
7      z[0] = n;
8      int l = 0,r = 0;
9      for(int i = 1;i < n;i++){
10         if (i < r) z[i] = min(r - i,z[i - 1]);
11         while(i + z[i] < n && s[z[i]] == s[i + z[i]]){
12             z[i]++;
13         }
14         if(i + z[i] > r){
15             l = i;
16             r = i + z[i];
17         }
18     }
19     return z;
20 }
21 // 先计算模式串 Z 函数, 再计算模式串与文本每个后缀的匹配长度。
22 void solve(){
23     string a,b;cin >> a >> b; int m = sz(b); int n = sz(a);
24     vector<int> z = get_z(b);
25     int ans = 0;
26     for(int i = 0;i < m;i++){
27         ans ^= (i + 1) * (z[i] + 1);
28     }
29     cout << ans << endl;
30     vector<int> t(n);
31     int l = 0,r = 0;
32     for(int i = 0;i < n;i++){
33         if (i < r) t[i] = min(r - i,z[i - 1]);
34         while(i + t[i] < n && t[i] < m && b[t[i]] == a[i + t[i]]){
35             t[i]++;

```

```

36         }
37         if(i + t[i] > r){
38             l = i;
39             r = i + t[i];
40         }
41     }
42     ans = 0;
43     for(int i = 0;i < n;i++){
44         ans ^= (i + 1) * (t[i] + 1);
45     }
46     cout << ans << endl;
47 }

```

String/Hash.cpp

```

1  #include<bits/stdc++.h>
2  using namespace std;
3  // 用单个模数计算字符串的多项式哈希值。
4  int B = 131,MOD = 1e9 + 7;
5  auto hash = [&](string s){
6      int res = 0;
7      for(int i = 0;i < s.length();i++){
8          res = ((int)res * B + s[i]) % MOD;
9      }
10     return res;
11 };
12 // 用两个模数计算字符串哈希, 降低冲突概率。
13 int B = 131,MOD1 = 1e9 + 7,MOD2 = 998244353;
14 auto hash = [&](string s,int MOD){
15     int res = 0;
16     for(int i = 0;i < s.length();i++){
17         res = ((int)res * B + s[i]) % MOD;
18     }
19     return res;
20 };
21 // 比较两个字符串的双模哈希是否不同; 不同则可判定字符串不同。
22 auto cmp = [&](string s,string t) -> bool {
23     return hash(s,MOD1) != hash(t,MOD1) || hash(s,MOD2) != hash(t,MOD2);
24 };
25 // 预处理前缀哈希和幂数组, 用于 O(1) 查询子串哈希。
26 vector<int> h(m + 1),p(m + 1);
27 p[0] = 1;
28 for(int i = 0;i < m;i++){
29     h[i + 1] = (h[i] * B + t[i]) % MOD;
30     p[i + 1] = p[i] * B % MOD;
31 }
32 // 返回半开区间 [l,r) 的子串哈希值。
33 auto get = [&](int l,int r){
34     return (h[r] - h[l] * p[r-l] % MOD + MOD) % MOD;
35 };

```

String/KMP.cpp

```

1  #include <bits/stdc++.h>
2  using namespace std;
3  #define int long long
4  #define endl "\n"
5  #define all(x) (x).begin(), (x).end()
6  #define rall(x) (x).rbegin(), (x).rend()
7  #define sz(x) (int)(x).size()
8  const int INF = (1LL << 62); const int MOD = 1e9 + 7; const int mod = 998244353; const int N = 10
00005;
9  string s, p; int nxt[N];
10 // 计算模式串的前缀函数, nxt[i] 是最长相等真前后缀长度。

```

```

11 void getNext()
12 {
13     int m = sz(p);
14     nxt[0] = 0;
15     for (int i = 1, j = 0; i < m; i++)
16     {
17         while (j && p[i] != p[j])
18             j = nxt[j - 1];
19         if (p[i] == p[j]) j++;
20         nxt[i] = j;
21     }
22 }
23 // 用前缀函数扫描文本, 输出模式串在 s 中所有 1-based 匹配起点。
24 void kmp()
25 {
26     int n = sz(s), m = sz(p);
27     for (int i = 0, j = 0; i < n; i++)
28     {
29         while (j && s[i] != p[j])
30             j = nxt[j - 1];
31         if (s[i] == p[j]) j++;
32         if (j == m)
33         {
34             cout << i - m + 2 << endl;
35             j = nxt[j - 1];
36         }
37     }
38 }
39 // 读入文本和模式串, 执行 KMP 匹配并输出前缀函数。
40 void solve()
41 {
42     cin >> s >> p;
43     getNext();
44     kmp();
45     for (int i = 0; i < sz(p); i++)
46         cout << nxt[i] << " ";
47     cout << endl;
48 }

```

String/Manacher.cpp

```

1 #include <bits/stdc++.h>
2 using namespace std;
3 // 用 Manacher 在线性时间内求字符串的最长回文子串长度。
4 void solve() {
5     string s; cin >> s; int n = s.length();
6     vector<int> d1(n + 1), d2(n + 1);
7     int l = 0, r = -1;
8     for (int i = 0; i < n; i++) {
9         if (i > r) d1[i] = 1;
10        else d1[i] = min(d1[r + 1 - i], r - i + 1);
11        while (i - d1[i] >= 0 && i + d1[i] < n && s[i - d1[i]] == s[i + d1[i]]) {
12            d1[i]++;
13        }
14        if (i + d1[i] - 1 > r) {
15            l = i - d1[i] + 1;
16            r = i + d1[i] - 1;
17        }
18    }
19    l = 0, r = -1;
20    for (int i = 0; i < n; i++) {
21        if (i > r) d2[i] = 0;
22        else d2[i] = min(d2[r + 1 - i + 1], r - i + 1);
23        while (i - d2[i] - 1 >= 0 && i + d2[i] < n && s[i - d2[i] - 1] == s[i + d2[i]]) {

```

```

24            d2[i]++;
25        }
26        if (i + d2[i] - 1 > r) {
27            l = i - d2[i];
28            r = i + d2[i] - 1;
29        }
30    }
31    int ans = 0;
32    for (int i = 0; i < n; i++) {
33        ans = max({ans, 2 * d1[i] - 1, 2 * d2[i]});
34    }
35    cout << ans << endl;
36 }

```

String/MinimalRotation.cpp

```

1 #include <bits/stdc++.h>
2 using namespace std;
3 // 用 Booth 思想求字典序最小循环表示, 并输出该旋转。
4 void solve() {
5     int n; cin >> n;
6     vector<int> a(2 * n);
7     for (int i = 0; i < n; i++) {
8         cin >> a[i];
9         a[i + n] = a[i];
10    }
11    int i = 0, j = 1, k = 0;
12    while (i < n && j < n && k < n) {
13        if (a[i + k] == a[j + k]) {
14            k++;
15            continue;
16        }
17        if (a[i + k] > a[j + k]) {
18            i += k + 1;
19            if (i == j) i++;
20        }
21        else if (a[i + k] < a[j + k]) {
22            j += k + 1;
23            if (i == j) j++;
24        }
25        k = 0;
26    }
27    int p = min(i, j);
28    for (int i = 0; i < n; i++) {
29        cout << a[p + i] << " ";
30    }
31    cout << endl;
32 }

```

String/SA.cpp

```

1 #include <bits/stdc++.h>
2 using namespace std;
3 // 倍增加计数排序构造后缀数组, 并输出每个后缀的排名顺序。
4 void solve() {
5     string s; cin >> s; int n = s.length(); s = " " + s;
6     //sa[i] pos of rank_i //rk[i] rank of pos_i
7     vector<int> sa(n + 1), rk(n + 2), tmp(n + 2);
8     vector<int> id(n + 1), cnt(max(300LL, n + 5));
9     int m = 300;
10    // 初始排名: 单个字符
11    for (int i = 1; i <= n; i++) {
12        rk[i] = s[i]; id[i] = i;
13    }
14    // 初始按字符计数排序

```

```

15     for(int i = 1;i <= n;i++)cnt[rk[i]]++;
16     for(int i = 1;i <= m;i++)cnt[i] += cnt[i - 1];
17     for(int i = n;i >= 1;i--)sa[cnt[rk[i]]--] = i;
18     for(int w = 1;;w <= 1){
19         int p = 0;
20         // 先按第二关键字排序
21         // i+w > n 的第二关键字为 0
22         for(int i = n - w + 1;i <= n;i++){
23             if (i >= 1) id[++p] = i;
24         }
25         // 其余位置的第二关键字是 rk[i+w]
26         // sa 已经按 rk 排序, 所以 sa[i]-w 就按第二关键字排好了
27         for(int i = 1;i <= n;i++){
28             if (sa[i] > w) id[++p] = sa[i] - w;
29         }
30         // 按第一关键字 rk[id[i]] 稳定计数排序
31         fill(cnt.begin(),cnt.end(),0);
32         for(int i = 1;i <= n;i++)cnt[rk[id[i]]]++;
33         for(int i = 1;i <= m;i++)cnt[i] += cnt[i - 1];
34         for(int i = n;i >= 1;i--)sa[cnt[rk[id[i]]]--] = id[i];
35         // 根据二元组重新离散化
36         tmp[sa[1]] = 1;p = 1;
37         for(int i = 2;i <= n;i++){
38             int x = sa[i - 1]; int y = sa[i]; int x1 = rk[x]; int x2 = (x + w <= n ? rk[x + w] :
0); int y1 = rk[y]; int y2 = (y + w <= n ? rk[y + w] : 0);
39             if(x1 == y1 && x2 == y2)tmp[y] = p;
40             else tmp[y] = ++p;
41         }
42         rk = tmp;m = p;
43         if (p == n) break;
44     }
45     for(int i = 1;i <= n;i++){
46         cout << sa[i] << " ";
47     }
48     cout << endl;
49 }
50 // 根据后缀数组和排名计算 height[i] = LCP(sa[i-1],sa[i])。
51 vector<int> height(n + 1);
52 int k = 0;
53 for(int i = 1;i <= n;i++){
54     int r = rk[i];
55     if(r == 1){
56         k = 0;
57         continue;
58     }
59     int j = sa[r - 1];
60     while(i + k <= n && j + k <= n && s[i + k] == s[j + k]){
61         k++;
62     }
63     height[r] = k;
64     if (k) k--;
65 }

```